

GamutX

CLI Tool Fundamentals

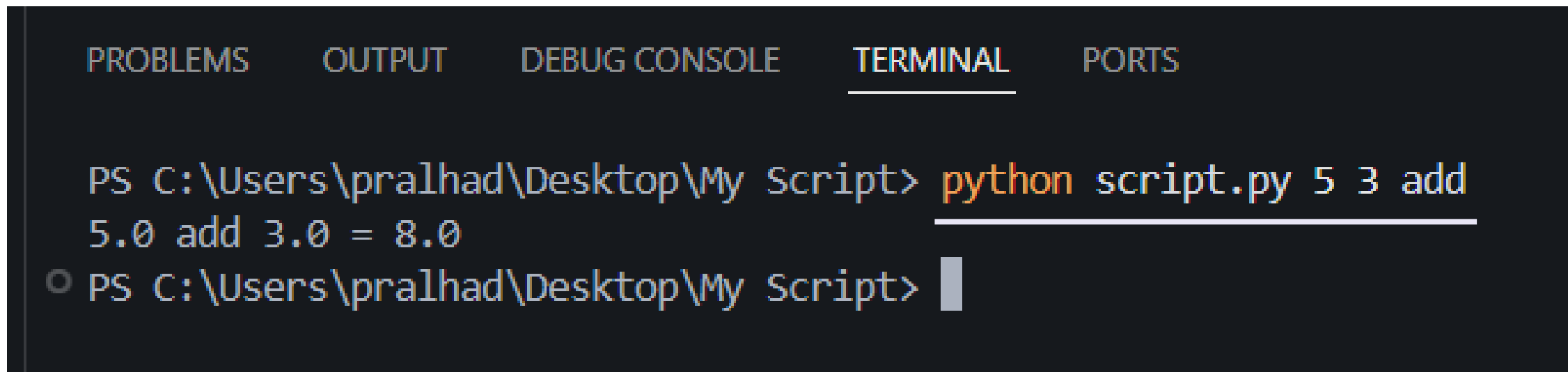
PYTHON SCRIPTING

Understanding the CLI

- What are Command Line Tools?
- Running Python Scripts in VS Code Terminal
- What is CMD (Command Prompt)?
- What Can We Do with CMD?
- What is PowerShell?
- CMD vs PowerShell

What are Command Line Tools?

Command Line Tools are simple **programs** or **scripts** that you run through the **terminal**.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

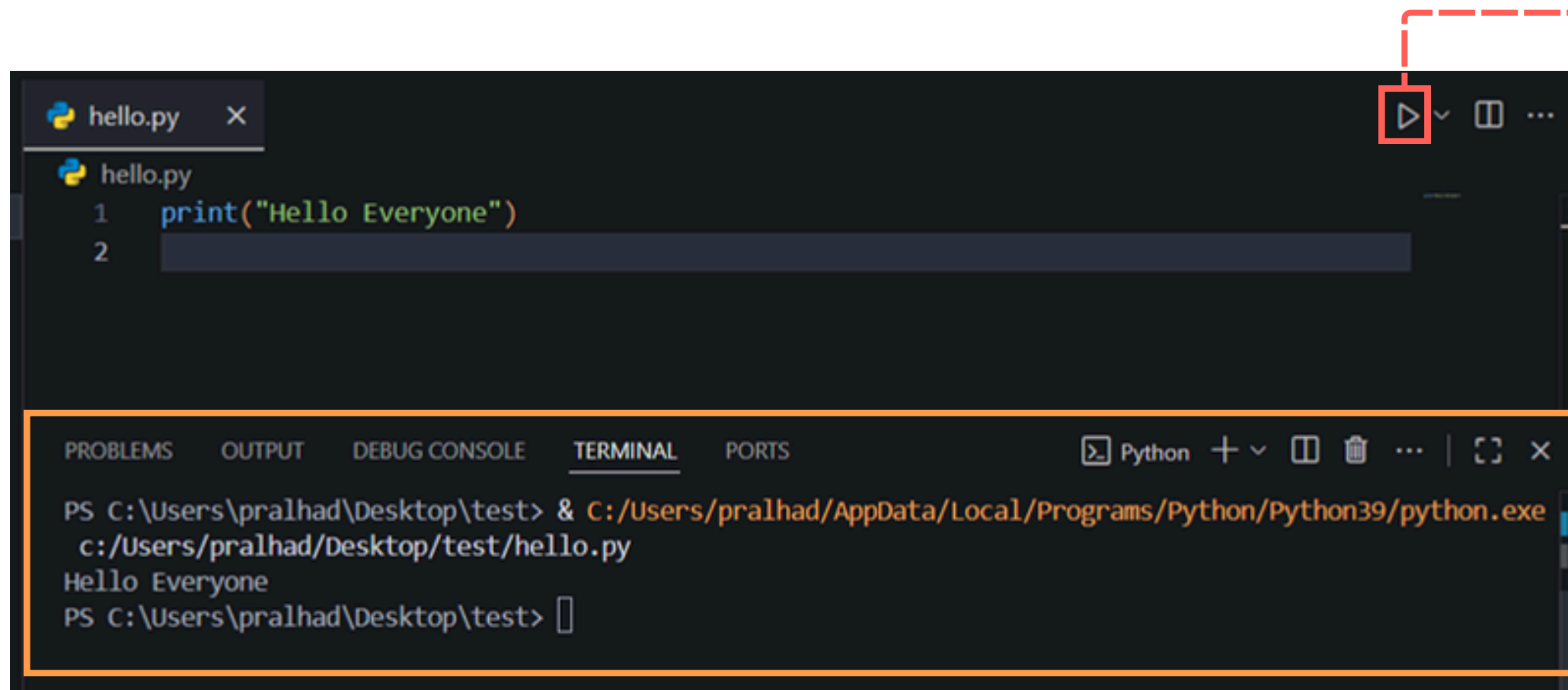
PS C:\Users\pralhad\Desktop\My Script> python script.py 5 3 add
5.0 add 3.0 = 8.0
○ PS C:\Users\pralhad\Desktop\My Script> |
```

Key characteristics:

- You type a command, press Enter, and the tool does its job.
- No need for graphical interfaces (no windows, buttons, etc.).
- Often written in Python, but could also be in Bash, Go, C++, etc.

Running Python Scripts in VS Code Terminal

- When you run a Python script in VS Code, by default, it uses the integrated terminal
- which just means the terminal that's built into the editor



The screenshot shows the VS Code interface. At the top, a file named 'hello.py' is open in the editor, containing the code: `1 print("Hello Everyone")` and an empty line `2`. In the top right corner of the editor, a red dashed box highlights the 'Run' button (a play icon). A red dashed arrow points from this button to the text 'When we run the hello.py Script.' Below the editor, the 'TERMINAL' tab is active, showing the command prompt output: `PS C:\Users\pralhad\Desktop\test> & C:/Users/pralhad/AppData/Local/Programs/Python/Python39/python.exe c:/Users/pralhad/Desktop/test/hello.py`, followed by the output `Hello Everyone` and a new prompt line `PS C:\Users\pralhad\Desktop\test>` . An orange dashed arrow points from the terminal output to the text 'Integrated VS Code Terminal'.

- When we run the hello.py Script.
- We'll see the output in VS Code Terminal

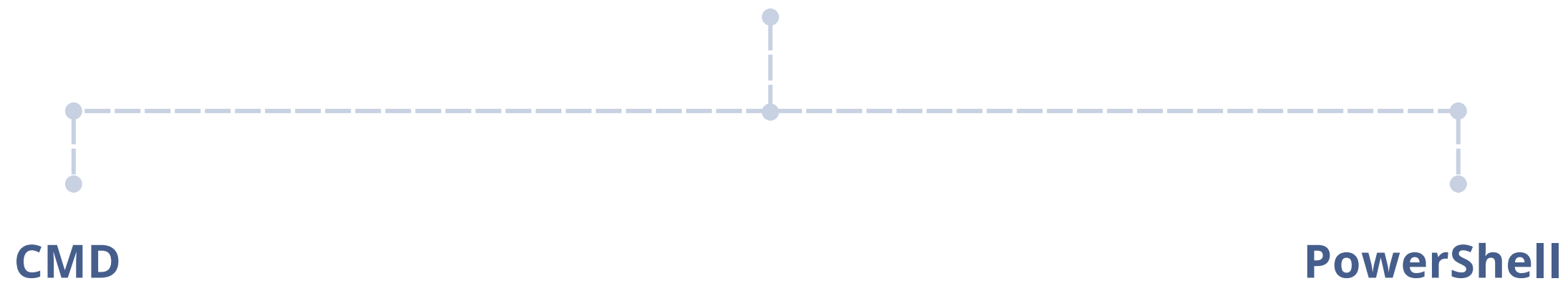
Integrated VS Code Terminal

Why is this convenient?

- We don't need to leave your editor to see the result—it's all in one place.

Q: But what if we want to run the **script outside of VS Code**?

👉 For that, we'll use the **system's built-in terminals** like or



```
Command Prompt
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.
C:\Users\pralhad>
```

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

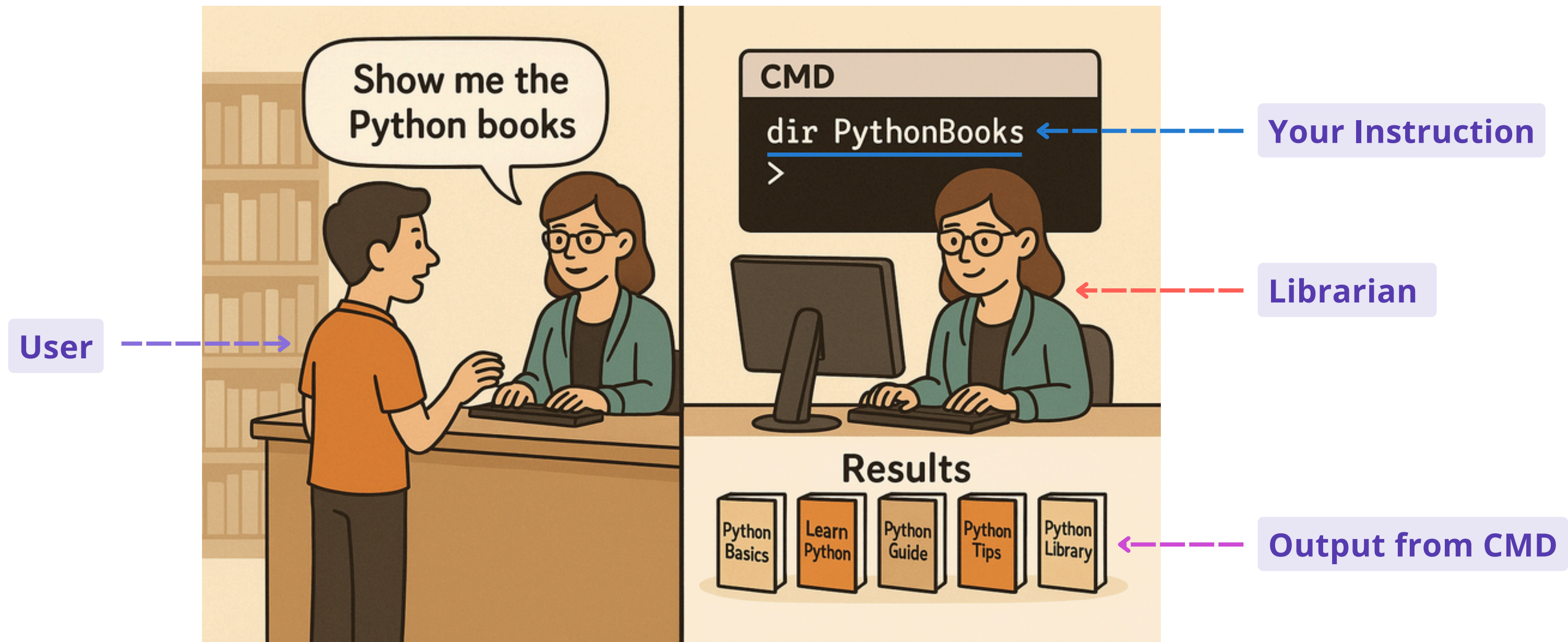
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\pralhad>
```

What is CMD (Command Prompt)?

- **CMD** stands for **Command Prompt**.
- It's a built-in terminal (or command-line interface) on **Windows** operating systems.

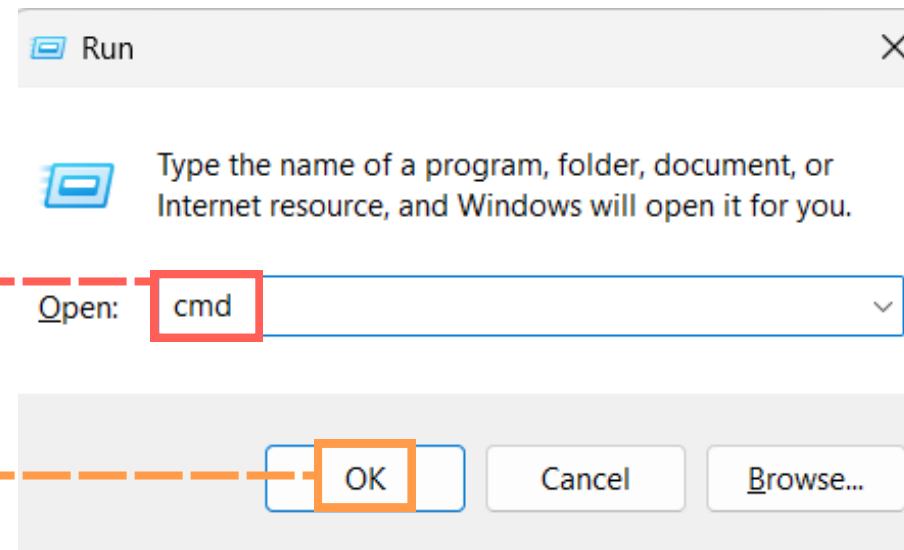
Think of it as a **text-based control panel** — you type instructions, and your computer carries them out.



How to open CMD:

✓ Option 1: Run Dialog

Press **Win + R** on your keyboard

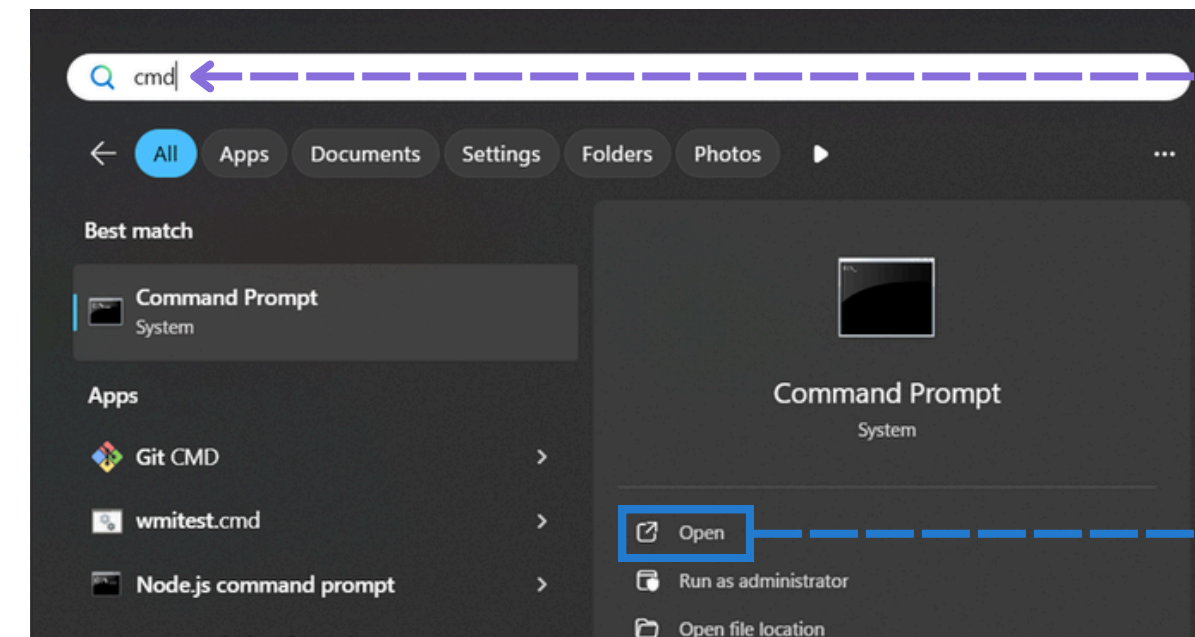


Type cmd

Press Ok

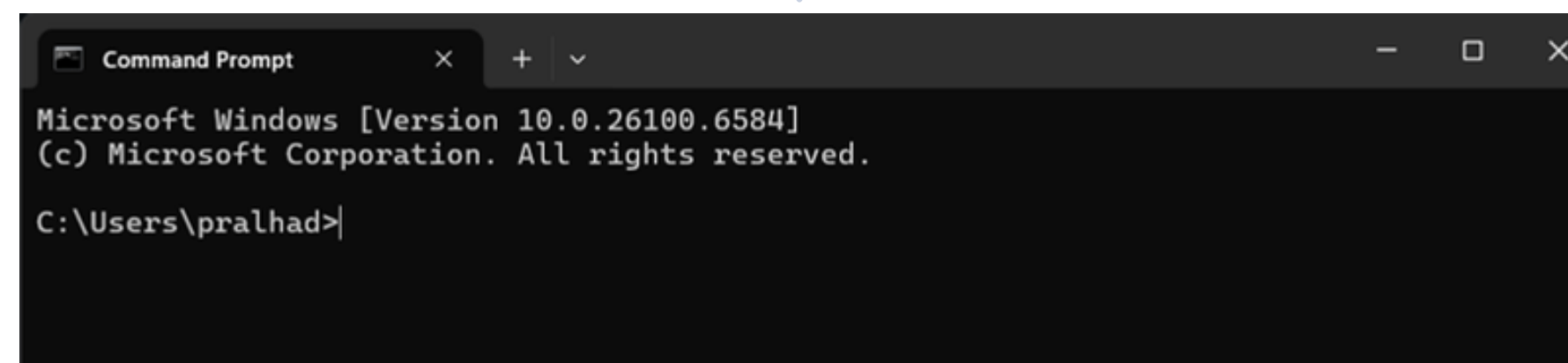
✓ Option 2: Start Menu

Click the Start Menu



Search CMD

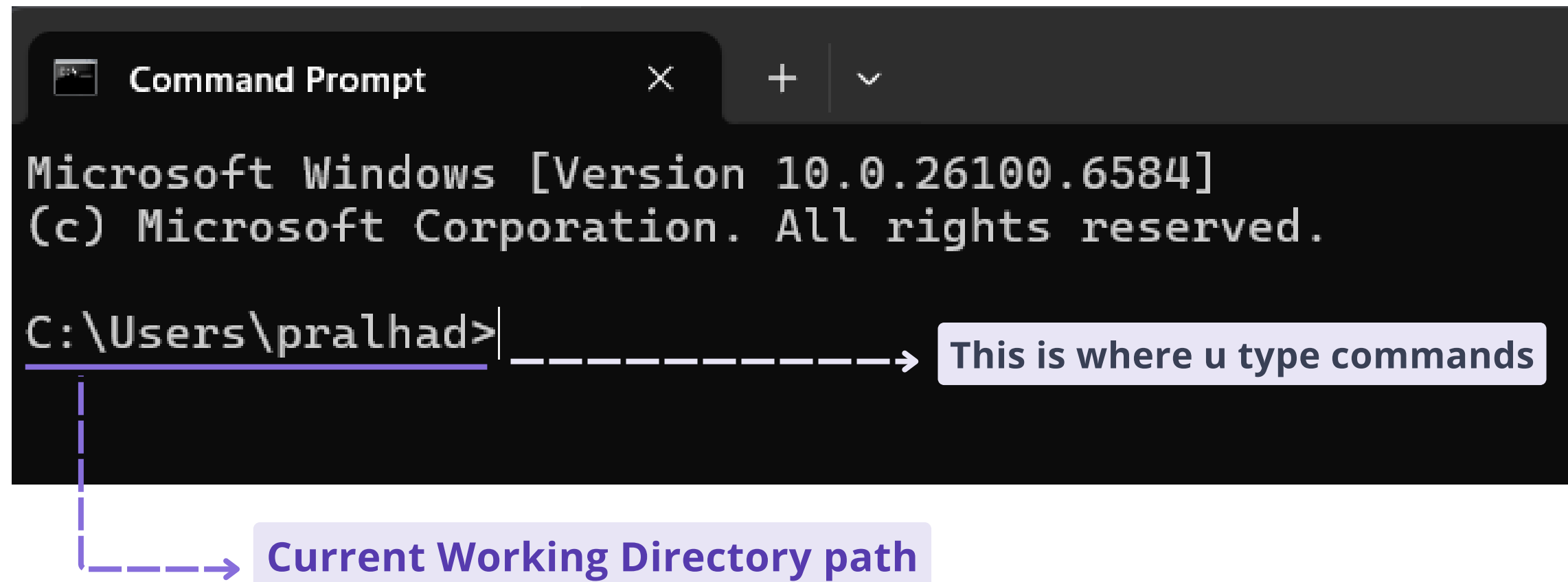
Click on Open



Note: At this stage, we're just exploring what CMD is. We're not running any scripts yet.

What We'll See

Once CMD is open:



The image shows a Windows Command Prompt window titled "Command Prompt". The window has a dark background and displays the following text:

```
Microsoft Windows [Version 10.0.26100.6584]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\pralhad>|
```

Two annotations are present:

- A dashed arrow points from the text "This is where u type commands" to the command prompt line.
- A dashed arrow points from the text "Current Working Directory path" to the path "C:\Users\pralhad" in the command prompt line.

What is PowerShell?

PowerShell is another **terminal** available on Windows.

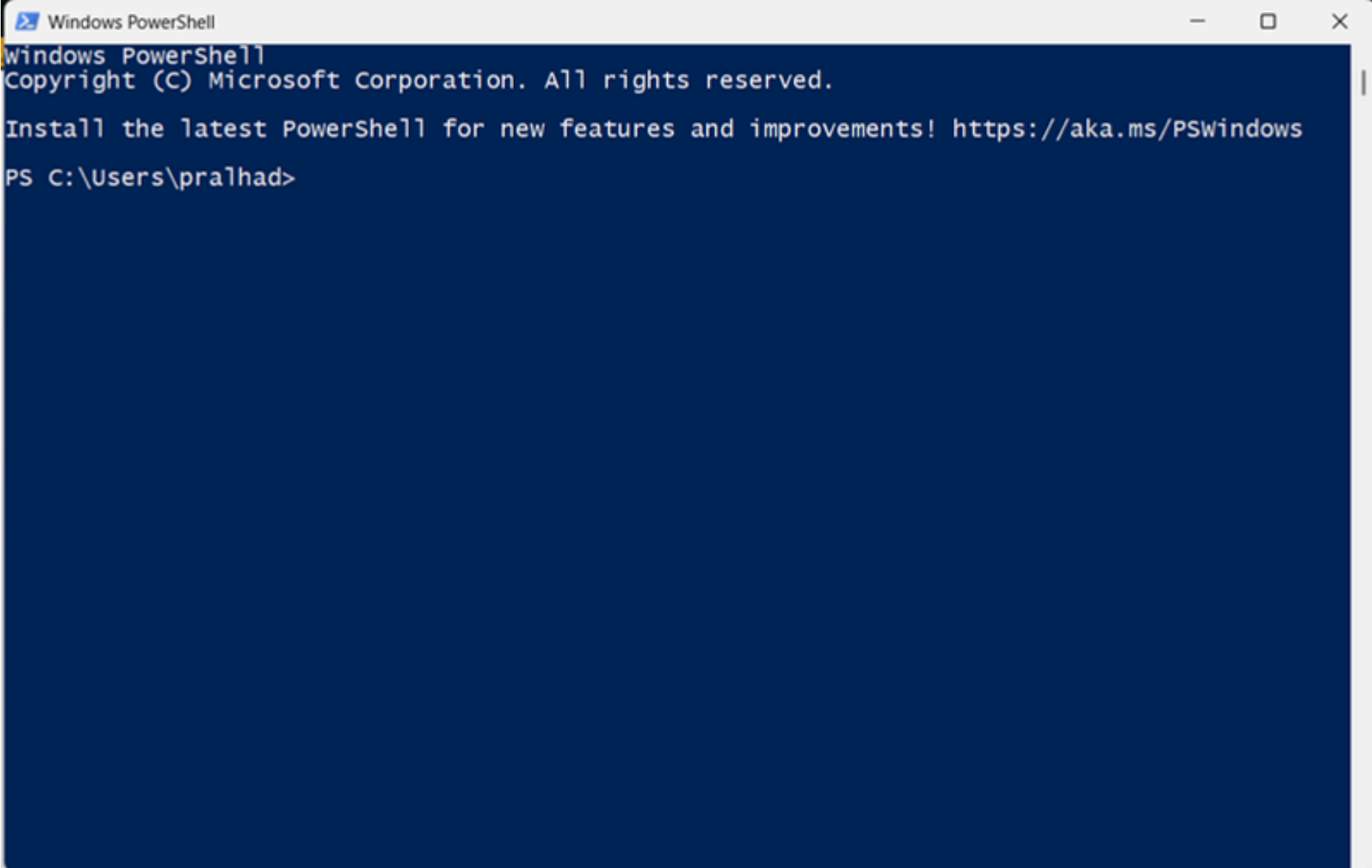
- It looks similar to **CMD** but comes with **more features**.
- The interface is **blue & Black** , so you'll instantly know you're in **PowerShell**.

How to open PowerShell:

- Press **Win + R**, type **powershell**, and hit **Enter**.
- Or, search for "**PowerShell**" in the **Start Menu**.

💡 Fun Fact:

- PowerShell is like CMD's "big brother."
- It's more advanced and user-friendly,
- especially for developers or system administrators.

A screenshot of a Windows PowerShell terminal window. The window has a title bar that says "Windows PowerShell". The background is dark blue. The text inside the window is white and reads: "Windows PowerShell", "Copyright (C) Microsoft Corporation. All rights reserved.", "Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows", and "PS C:\Users\pralhad>".

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\pralhad>
```

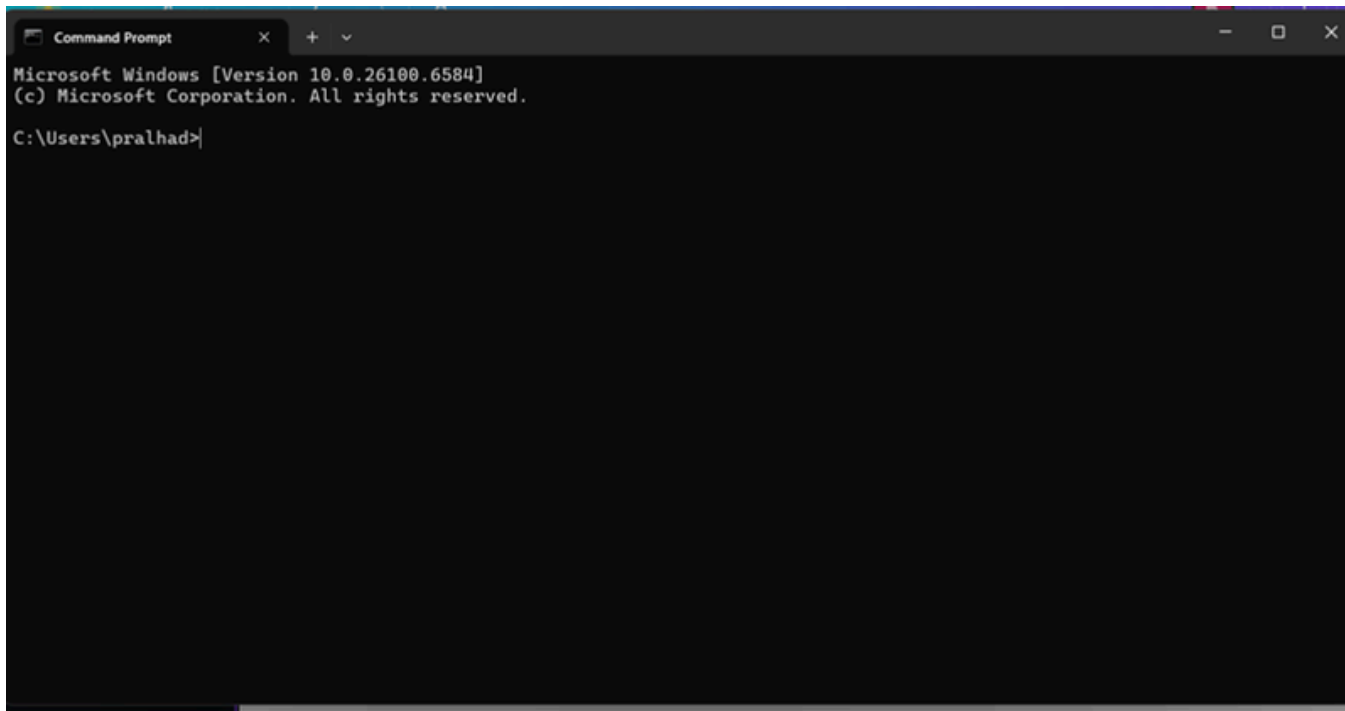



If CMD and PowerShell are so similar,
why do we have both? Can't we just use one?



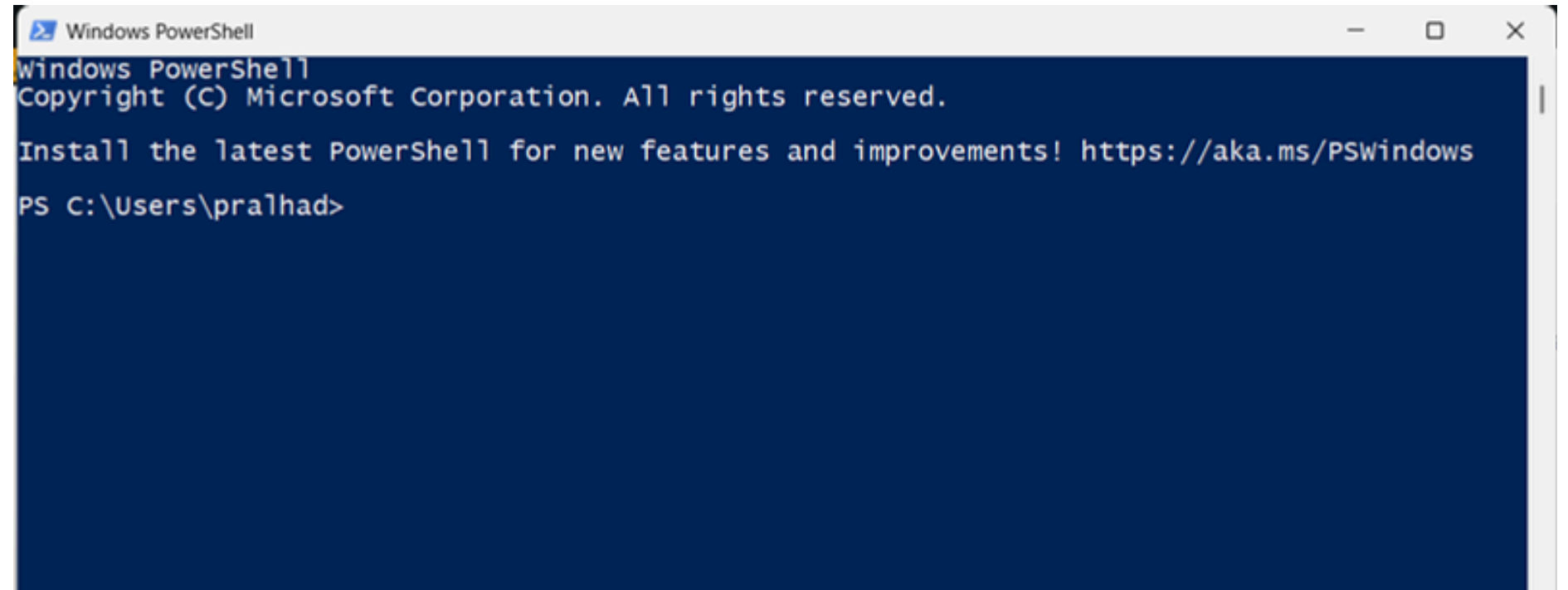
CMD vs PowerShell

CMD

A screenshot of the Windows Command Prompt (CMD) window. The title bar reads "Command Prompt". The window content shows the Microsoft Windows version (10.0.26100.6584) and copyright information. The prompt is at the C:\Users\pralhad> location.

```
Command Prompt
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.
C:\Users\pralhad>
```

Powershell

A screenshot of the Windows PowerShell window. The title bar reads "Windows PowerShell". The window content shows the Windows PowerShell version and copyright information, followed by a message to install the latest PowerShell for new features and improvements, with a link to https://aka.ms/PSWindows. The prompt is at the PS C:\Users\pralhad> location.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\pralhad>
```

- **CMD** is the **older**, simpler terminal.
 - Great for **basic tasks** like
 - navigating folders,
 - listing files, and running batch files.

- **PowerShell** is **newer** and **more powerful**.
 - Supports everything CMD can do, plus:
 - Scripting with variables and loops
 - Advanced automation
 - Accessing Windows system APIs

Terminal Basics

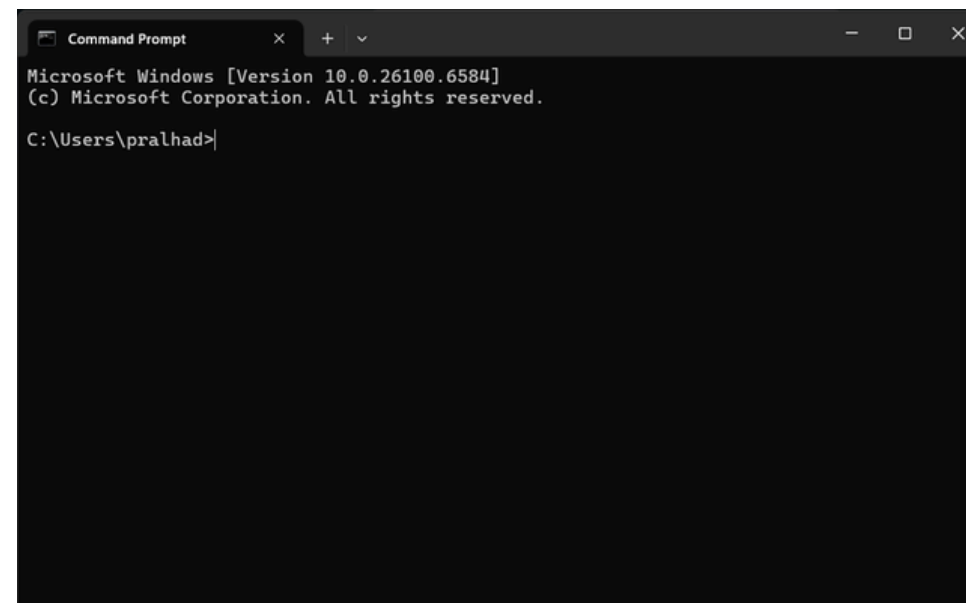
- What is Terminal ?
- What Can We Do with CMD?
- Why Do We Use the Terminal?

What is Terminal ?

A text-based interface to interact with your operating system.

Example of **Terminals**

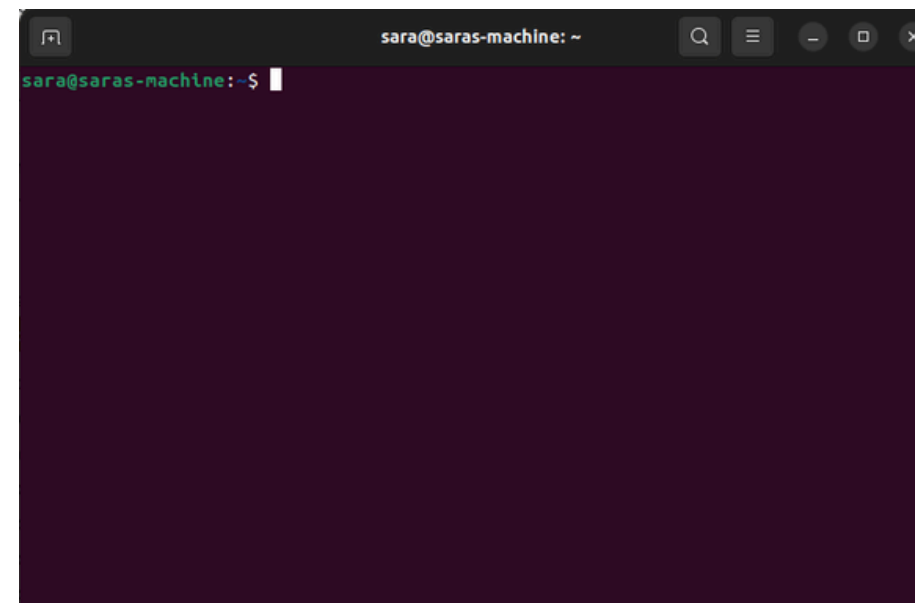
Windows

A screenshot of the Windows Command Prompt window. The title bar says "Command Prompt". The text inside shows the Windows version and copyright information, followed by the prompt "C:\Users\pralhad>".

```
Command Prompt
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.
C:\Users\pralhad>
```

Command Prompt (CMD)
or PowerShell

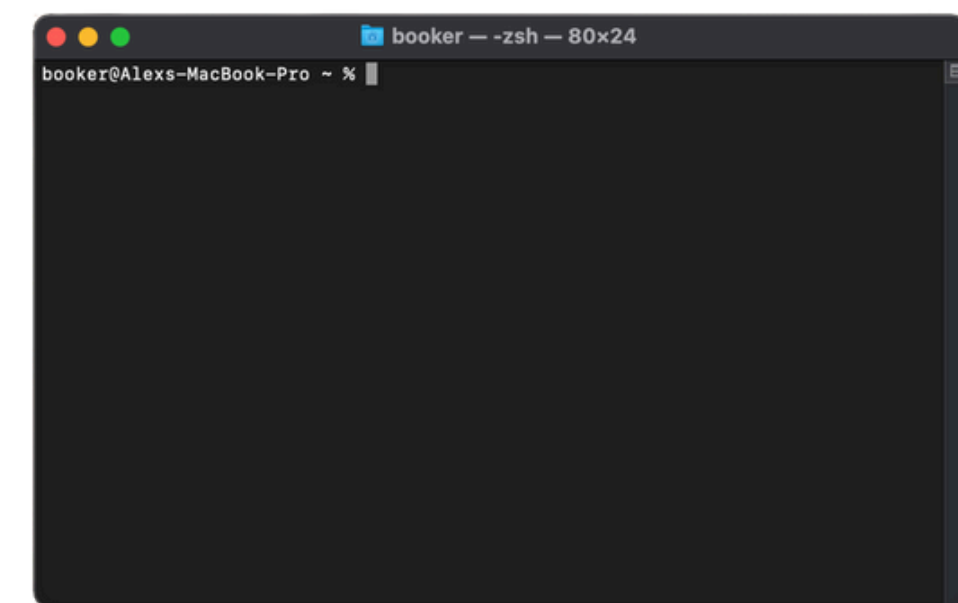
Linux

A screenshot of a Linux terminal window. The title bar shows "sara@saras-machine: ~". The prompt is "sara@saras-machine: ~\$".

```
sara@saras-machine: ~
sara@saras-machine: ~$
```

bash

macOS

A screenshot of a macOS terminal window. The title bar shows "booker - zsh - 80x24". The prompt is "booker@Alexs-MacBook-Pro ~ %".

```
booker - zsh - 80x24
booker@Alexs-MacBook-Pro ~ %
```

zsh

What Can We Do with CMD?

Task

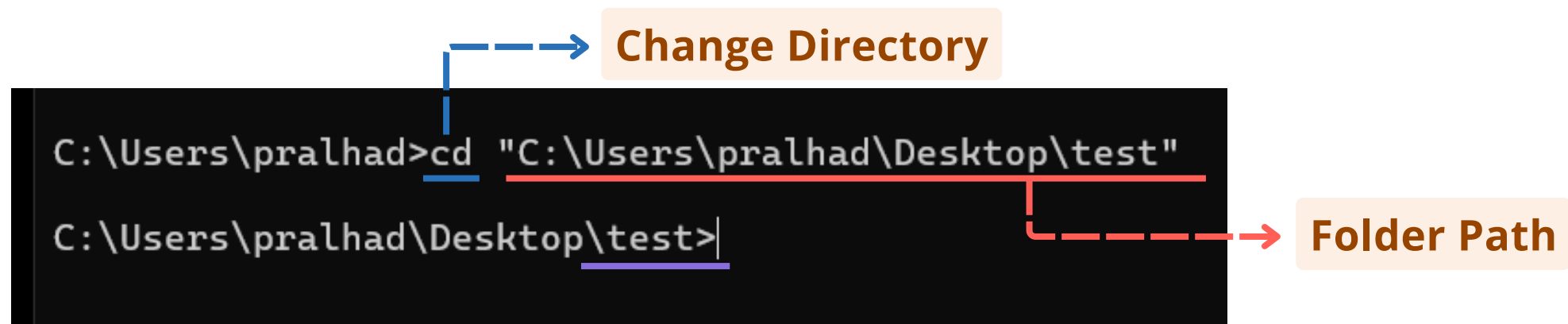
Example Command

Purpose

- | | | |
|---------------------------------|-----------------------------------|---|
| • Navigate folders | → <code>cd Desktop\scripts</code> | → Move into a specific folder |
| • View files in a folder | → <code>dir</code> | → List files and subfolders |
| • Run a Python script | → <code>python hello.py</code> | → Execute a script directly |
| • Create a directory | → <code>mkdir new_project</code> | → Make a new folder inside the current path |
| • Run an executable | → <code>MySoftware.exe</code> | → Launch a program from its folder |

Navigate Folders

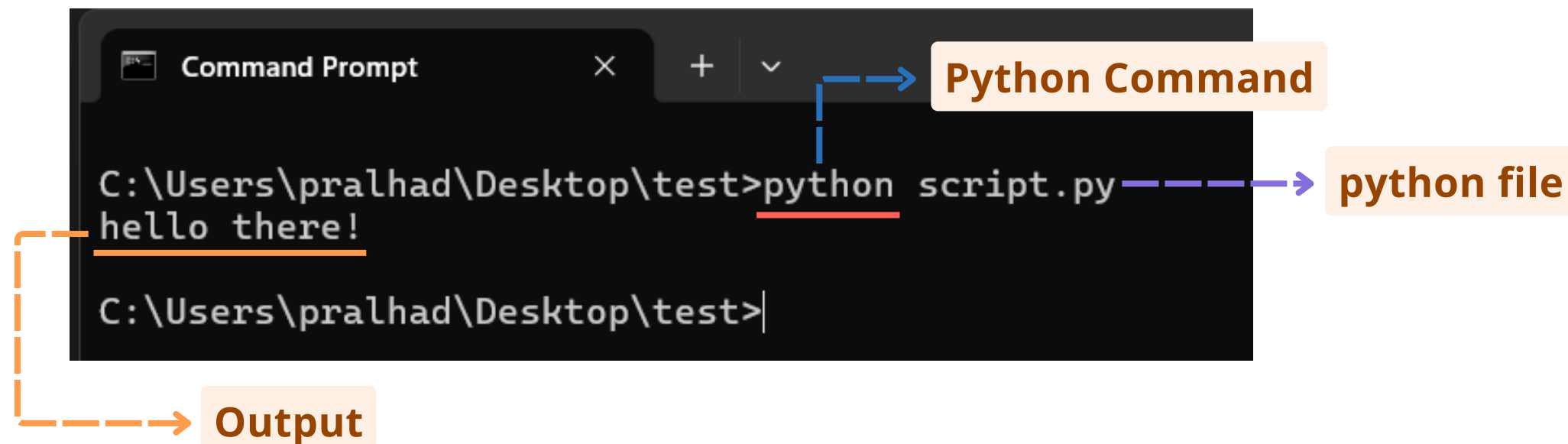
Move into a specific folder using the cd (change directory) command.



```
C:\Users\pralhad>cd "C:\Users\pralhad\Desktop\test"  
C:\Users\pralhad\Desktop\test>
```

Run a Python Scripters

Execute a Python script directly from the Command Prompt.



```
Command Prompt  
C:\Users\pralhad\Desktop\test>python script.py  
hello there!  
C:\Users\pralhad\Desktop\test>
```

View Files in a Folder

Command used to list all files and folders in the current directory.

```
Command Prompt
C:\Users\pralhad\Desktop\test>dir
Volume in drive C is OS
Volume Serial Number is C031-AD22

Directory of C:\Users\pralhad\Desktop\test

30-09-2025  16:51    <DIR>          .
29-09-2025  18:49    <DIR>          ..
29-09-2025  20:23    <DIR>        assets
29-09-2025  20:50    <DIR>        bin
30-09-2025  16:40      20 example.bin
30-09-2025  16:40     436 script.py
30-09-2025  16:52      59 t.py
                                515 bytes
               3 File(s)
               4 Dir(s)  792,799,506,432 bytes free

C:\Users\pralhad\Desktop\test>
```

. (dot)→ Refers to the current directory itself.

.. (double dot)→ Refers to the parent directory (one level up).

Numbers before the filenames (like 20, 436, 59) → File sizes in bytes.

Create a Directory

Use to Make a new folder inside the current directory.

```
Command Prompt
C:\Users\pralhad\Desktop>mkdir testFolder
C:\Users\pralhad\Desktop>dir
Volume in drive C is OS
Volume Serial Number is C031-AD22

Directory of C:\Users\pralhad\Desktop

09-2025  20:24    <DIR>          .
09-2025  14:03    <DIR>          ..
09-2025  14:31    <DIR>          experiments
09-2025  19:20    <DIR>          gamutx-VSD-MASTER
09-2025  14:58    <DIR>          My Script
09-2025  21:21    <DIR>          rango se bhari
09-2025  21:27      123,055,909 rango se bhari.zip
09-2025  18:42      195 scrippt.py
09-2025  16:51    <DIR>          test
09-2025  20:24    <DIR>          testFolder
                2 File(s)      123,056,104 bytes
                8 Dir(s)  792,795,688,960 bytes free

C:\Users\pralhad\Desktop>
```

used to create a new folde

Name of Folder

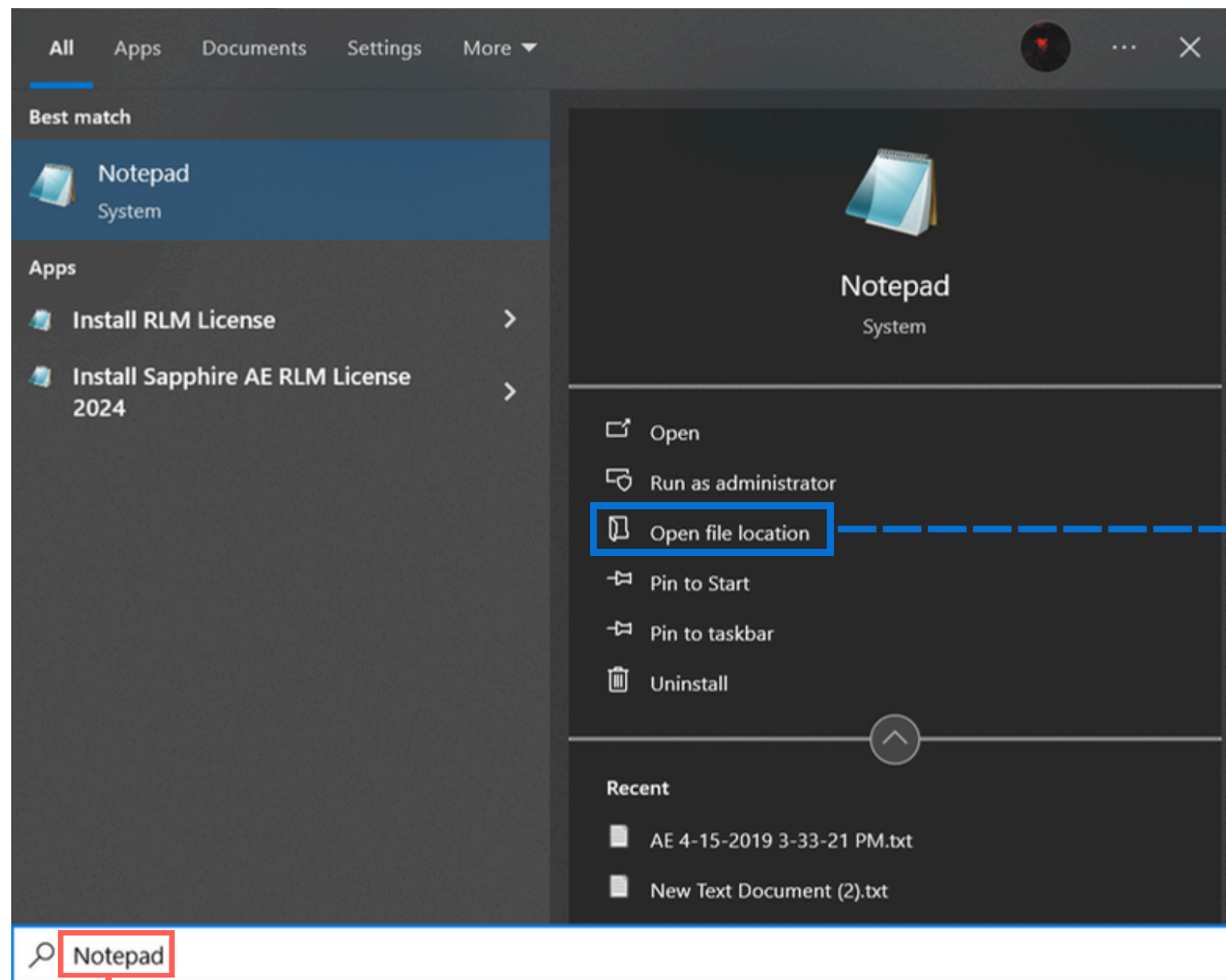
List all files & Folder to check it created or not

Folder Created

Run an Executable File

Find the .exe File Location

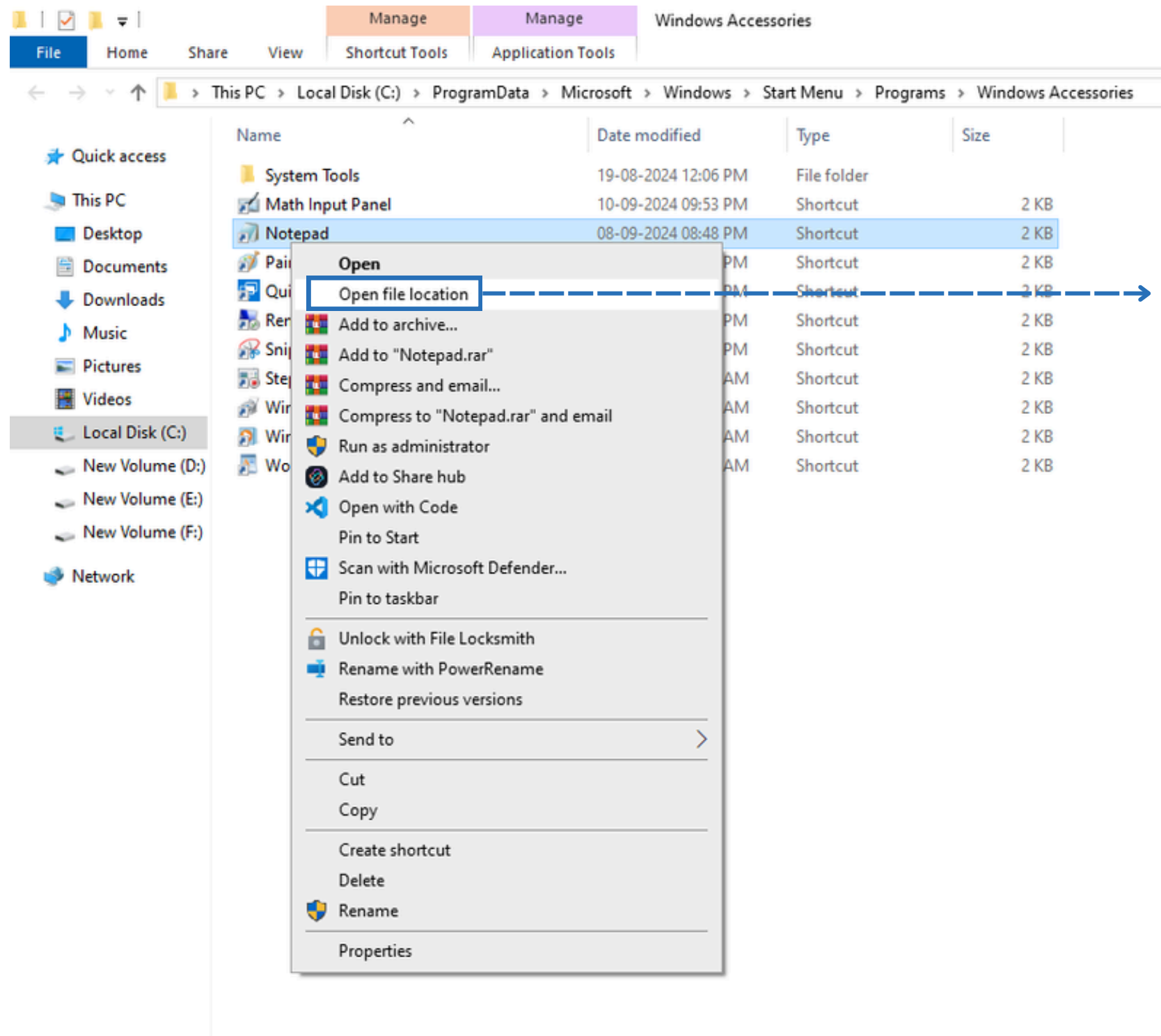
- Click on the Start Menu, search for Notepad.



Right-click → Open file location.

Search notepad

In the folder,



right-click the shortcut

*Open file location again. where **exe** file located*

Copy the path from the address bar, for Notepad it is usually: C:\Windows\System32

File

Home

Share

View

Application Tools

Manage

system32

←

→

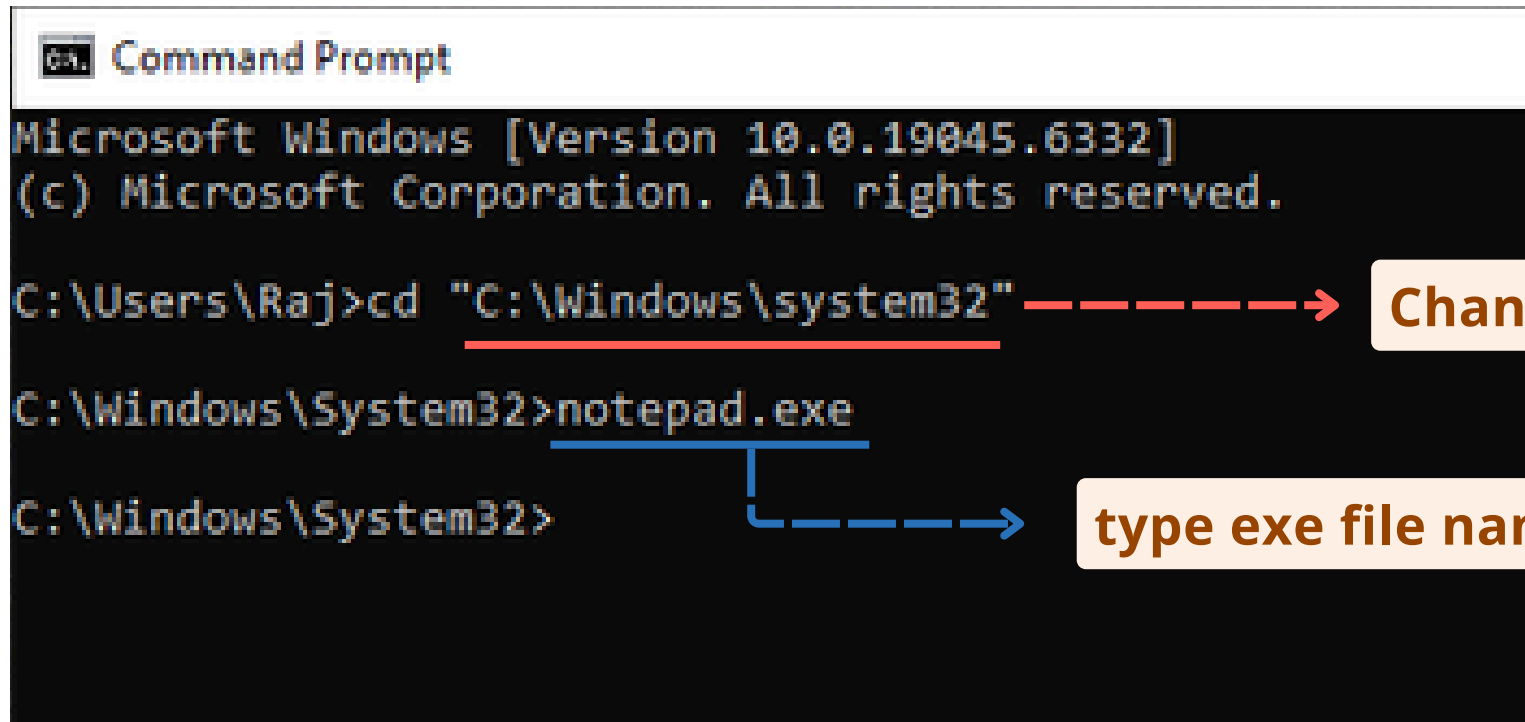
⌵

⬆

C:\Windows\system32

Name	Date modified	Type	Size
NmaDirect.dll	17-05-2025 05:09 PM	Application extension	261 KB
NOISE.DAT	07-12-2019 02:42 PM	DAT File	1 KB
normaliz.dll	19-08-2024 12:01 PM	Application extension	6 KB
normidna.nls	07-12-2019 02:38 PM	NLS File	80 KB
normnfc.nls	07-12-2019 02:38 PM	NLS File	56 KB
normnfd.nls	07-12-2019 02:38 PM	NLS File	50 KB
normnfkc.nls	07-12-2019 02:38 PM	NLS File	78 KB
normnfkd.nls	07-12-2019 02:38 PM	NLS File	72 KB
notepad.exe	17-05-2025 05:10 PM	Application	196 KB
NotificationController.dll	17-05-2025 05:09 PM	Application extension	1,200 KB
NotificationControllerPS.dll	17-05-2025 05:09 PM	Application extension	383 KB
notificationplatformcomponent.dll	19-08-2024 12:00 PM	Application extension	45 KB
npmproxy.dll	19-08-2024 12:01 PM	Application extension	48 KB
NPSM.dll	17-05-2025 05:09 PM	Application extension	198 KB
NPSMDesktopProvider.dll	17-05-2025 05:09 PM	Application extension	209 KB
nrpsrv.dll	07-12-2019 02:38 PM	Application extension	19 KB
nshhttp.dll	19-08-2024 12:02 PM	Application extension	51 KB
nshipsec.dll	19-08-2024 12:01 PM	Application extension	374 KB
nshwfp.dll	24-06-2025 11:49 AM	Application extension	772 KB

Open CMD and Launch Notepad



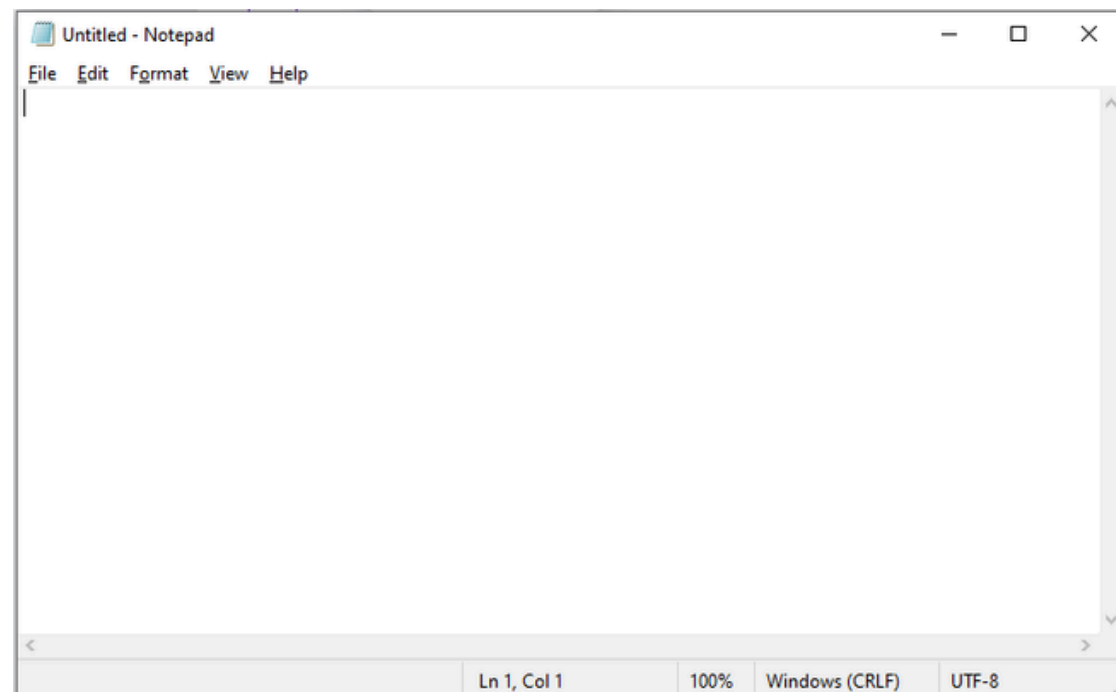
```
Command Prompt
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Raj>cd "C:\Windows\system32"
C:\Windows\System32>notepad.exe
C:\Windows\System32>
```

Change Directory to same location where exe file is

type exe file name and press enter to launch it

Output



Notepad will launch immediately.

Why Do We Use the Terminal?

Speed & Efficiency

- We can perform complex operations with a **single line of code**.
- Quicker than navigating with mouse clicks for repetitive tasks.

Essential for Development & Pipelines

- Used heavily in
 - **VFX pipelines, game engines, data processing, and build systems.**
- Automate repetitive tasks:
 - rendering, file transfers, backups, etc.

Automation

- Run scripts
- Trigger builds
- Deploy applications
- Cleanup projects

Control & Power

- Access hidden system settings
- Run commands with admin privileges
- Troubleshoot issues via logs and debug tools

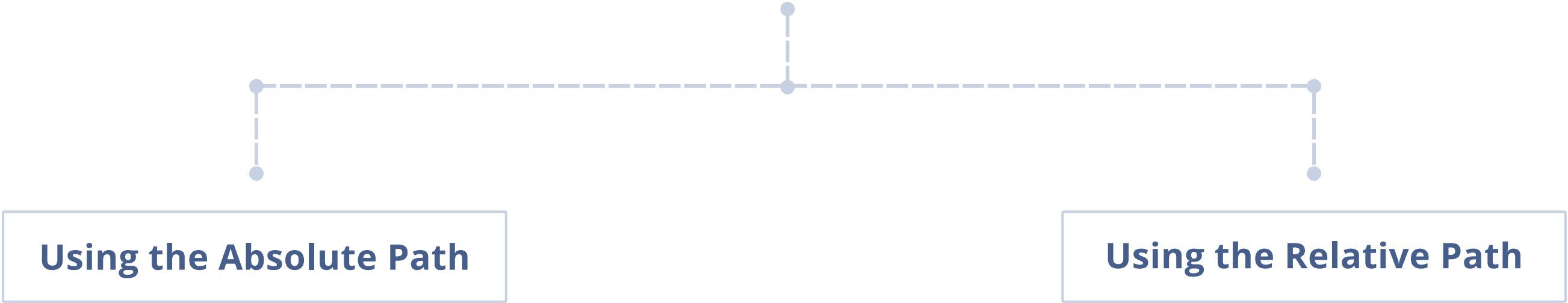


Can We Run Python Script in Both ?

Running Python Scripts in Terminal

- Using the Absolute Path
- What If the Path Has Spaces?
- Using the Relative Path

Two Ways to Run a **Python Script** from Terminal



Using the Absolute Path

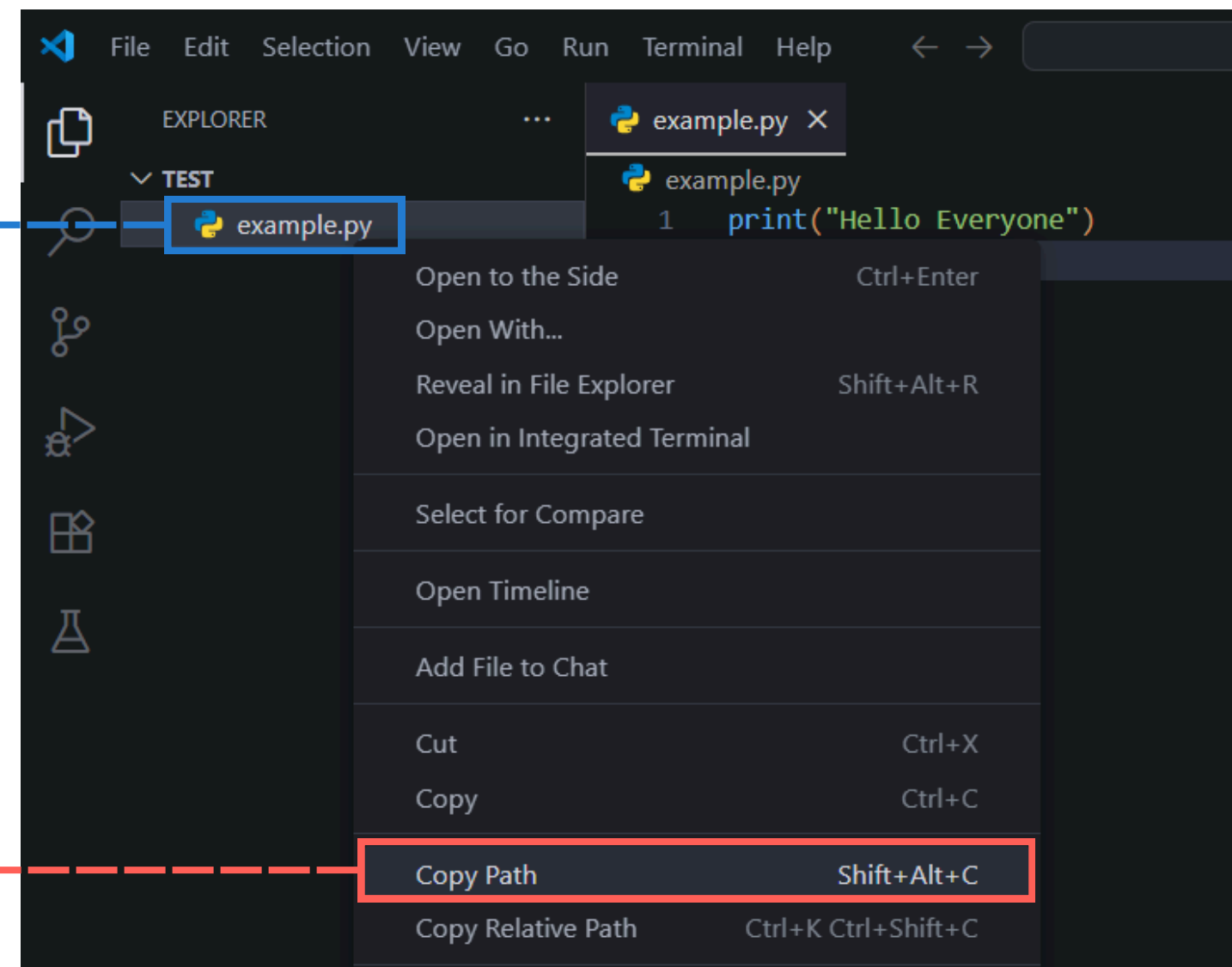
Using the Relative Path

Using the Absolute Path

- An **absolute path** is the full address of a file on your system.
- It starts from the root of the drive (like C:\) and includes **every folder** leading to the file.

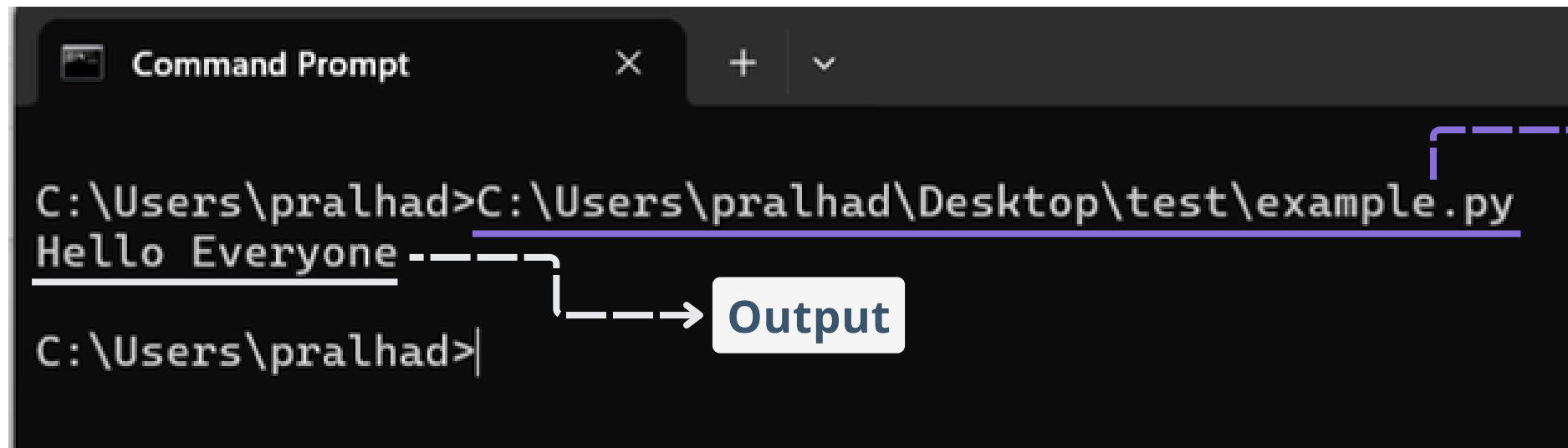
Create a Python file named example.py with this content:

locate file and right click on it



Click to Copy Absolute path

Open PowerShell (or CMD)



The screenshot shows a Windows Command Prompt window with the title 'Command Prompt'. The command prompt shows the current directory as 'C:\Users\pralhad' and the user has entered the command 'C:\Users\pralhad\Desktop\test\example.py'. The output of the command is 'Hello Everyone'. A dashed line connects the output to a label 'Output'.

```
Command Prompt
C:\Users\pralhad>C:\Users\pralhad\Desktop\test\example.py
Hello Everyone
C:\Users\pralhad>
```

Paste Path and press Enter

💡 Why It Work ?

Because our system is **configured to recognize .py files as Python scripts**.

- ✓ Python is installed
- ✓ The **.py** extension is **associated** with Python in Windows
- ✓ Python's location is registered in the **system PATH**
- ✓ **.py** is included in the PATHEXT environment variable

So **CMD** treats **.py** files like **.exe** or **.bat** files and automatically runs them using Python.

When This Will Not Work

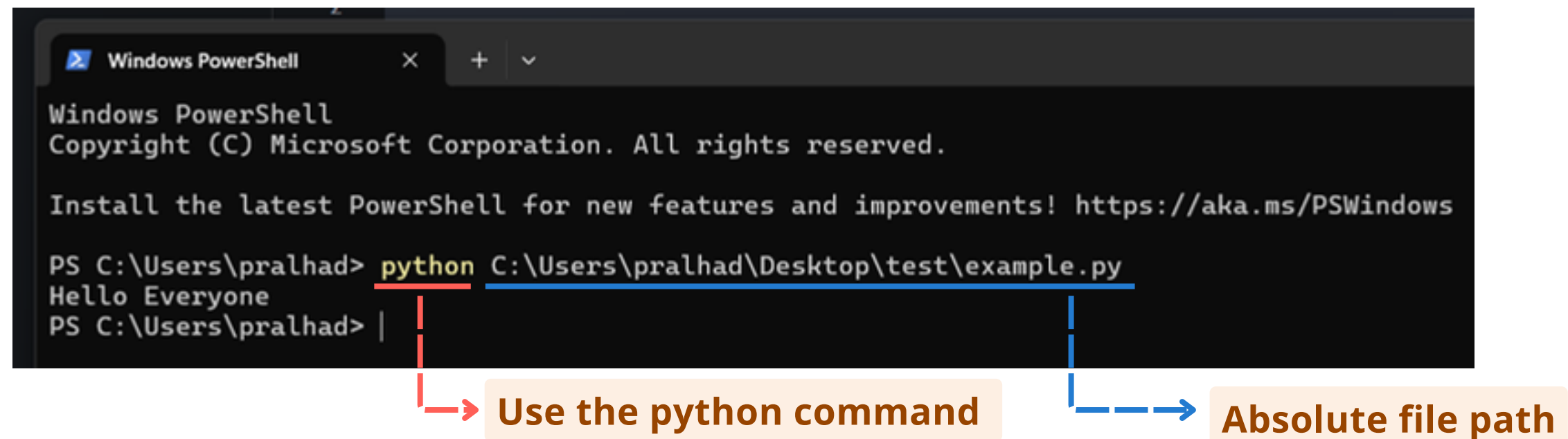
This same command may **fail** on other systems if:

- Python is not installed
- Python was installed but not added to the **PATH**
- **.py** is not registered with Python

Common Error:

```
'C:\path\to\example.py' is not recognized as an internal or external command...
```

Best Practice



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\pralhad> python C:\Users\pralhad\Desktop\test\example.py
Hello Everyone
PS C:\Users\pralhad> |
```

The screenshot shows a Windows PowerShell terminal window. The command `python C:\Users\pralhad\Desktop\test\example.py` is entered and executed successfully, outputting `Hello Everyone`. A red arrow points from the word `python` to an orange box labeled `Use the python command`. A blue arrow points from the file path `C:\Users\pralhad\Desktop\test\example.py` to an orange box labeled `Absolute file path`.

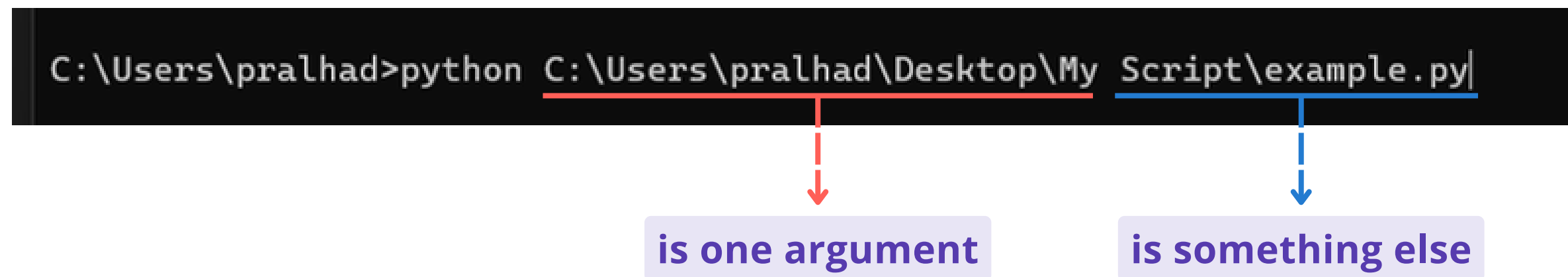
What If the Path Has Spaces?

```
Command Prompt
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pralhad> python C:\Users\pralhad\Desktop\My Script\example.py
python: can't open file 'C:\\Users\\pralhad\\Desktop\\My': [Errno 2] No such file or directory

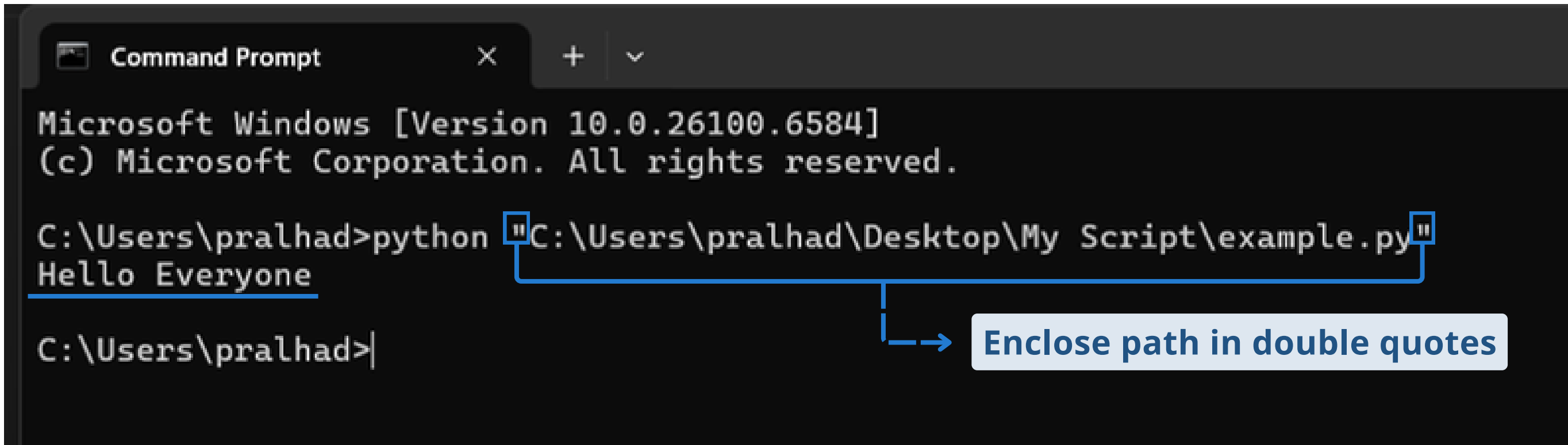
C:\Users\pralhad>
```

The Terminal thinks



It splits at the space, breaking the command

✓ The Fix: Use Double Quotes Around the Full Path



```
Command Prompt
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pralhad>python "C:\Users\pralhad\Desktop\My Script\example.py"
Hello Everyone

C:\Users\pralhad>
```

Enclose path in double quotes

Always Use Quotes If:

- Any folder in the path has a space
- You're copying a path from File Explorer that contains spaces
- Even if the path **doesn't** have spaces, using quotes is still safe:

Summary

What You Type

Will It Work?

Why / Why Not

- `C:\path\to\script.py`

→ ✓

→ Only if .py is registered with Python

- `C:\path with spaces\script.py`

→ ✗

→ Fails due to unquoted path

- `python C:\path\to\script.py`

→ ✓

→ Works if Python is in PATH

- `python C:\path with spaces\script.py`

→ ✗

→ Fails due to unquoted path

- `python "C:\path with spaces\script.py"`

→ ✓

→ Safe and recommended in all cases

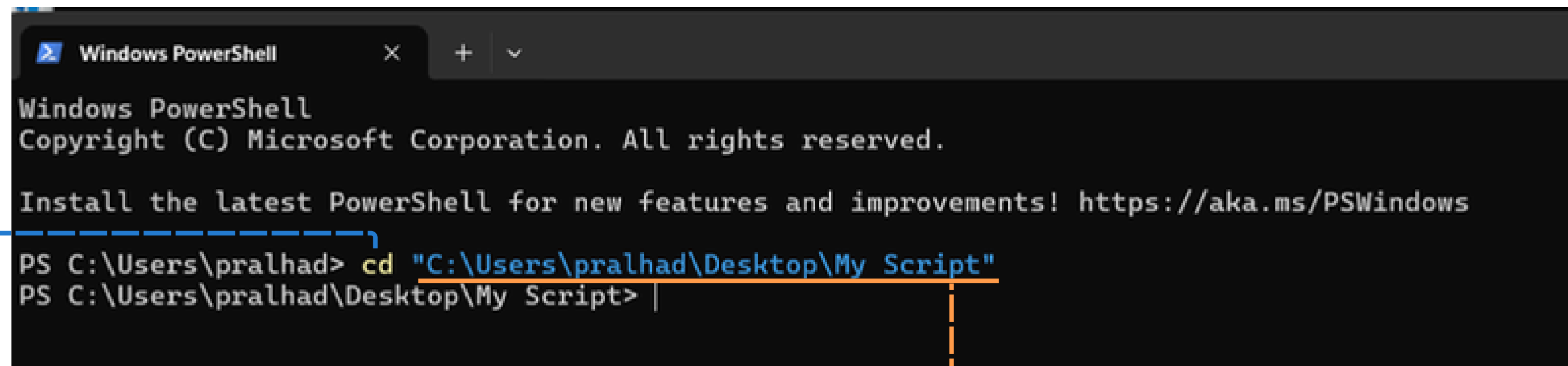
Using the Relative Path

- A relative path means you're running a script relative to your current location in the terminal.
- You don't need the full file path — just change your terminal's working directory to the folder where the file lives.

Let's say we have this file: "C:\Users\pralhad\Desktop\My Script"

Change to the Script's Folder

1. Open PowerShell.
2. Type `cd` (short for change directory) followed by a space.
3. Paste the **copied path** into PowerShell.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

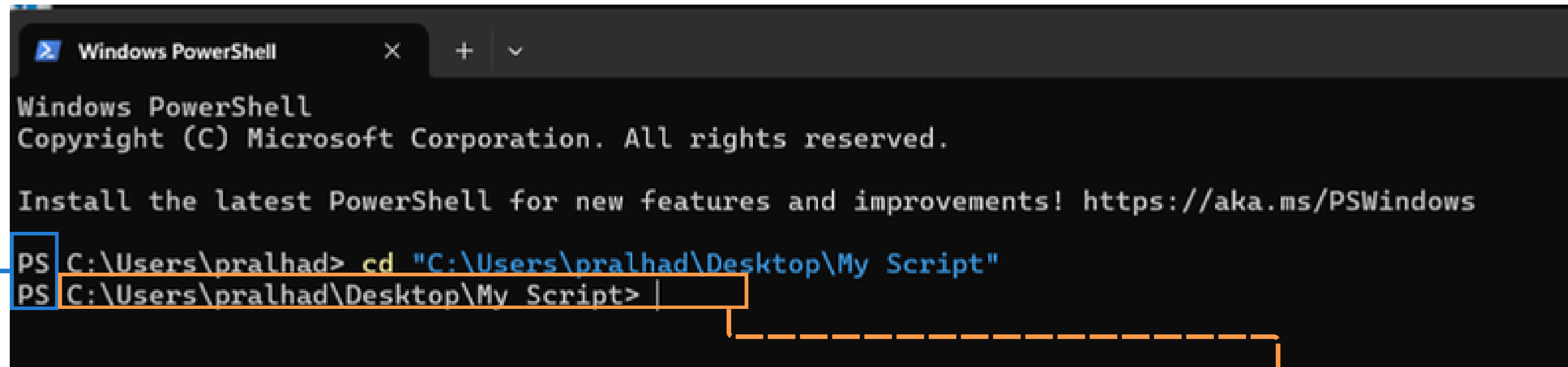
PS C:\Users\pralhad> cd "C:\Users\pralhad\Desktop\My Script"
PS C:\Users\pralhad\Desktop\My Script> |
```

The screenshot shows a Windows PowerShell terminal window. The title bar says "Windows PowerShell". The prompt is "PS C:\Users\pralhad>". The command entered is "cd \"C:\Users\pralhad\Desktop\My Script\"". The prompt has changed to "PS C:\Users\pralhad\Desktop\My Script>". A blue dashed arrow points from the "cd" command to the text "use to change Directory". An orange dashed arrow points from the double quotes around the path to the text "enclose it in double quotes".

use to change Directory

enclose it in double quotes

Confirm the Terminal Location



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\pralhad> cd "C:\Users\pralhad\Desktop\My Script"
PS C:\Users\pralhad\Desktop\My Script> |
```

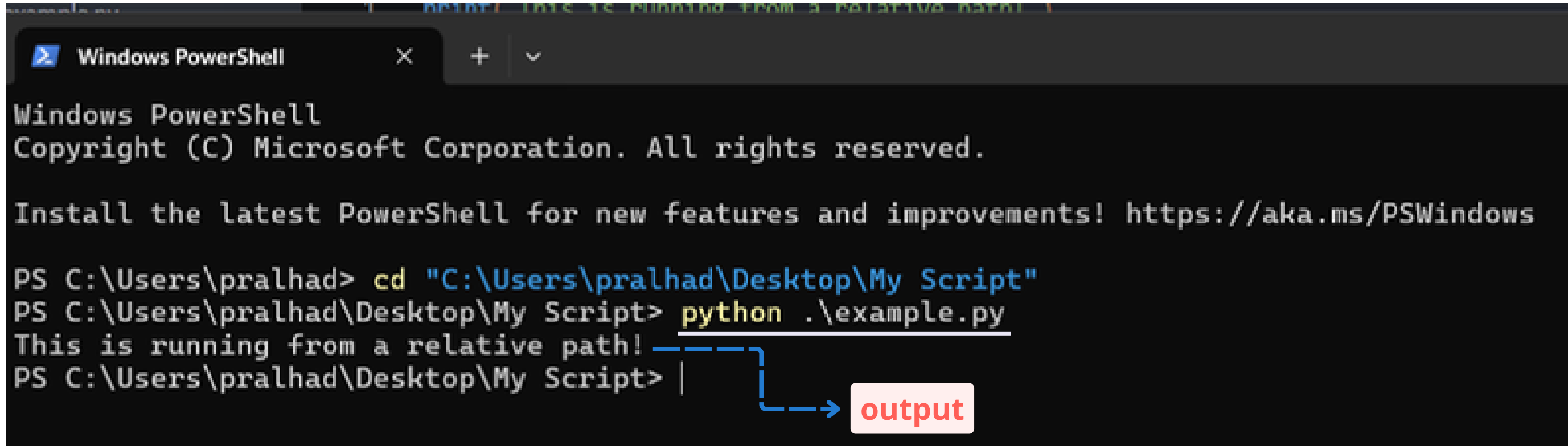
The screenshot shows a Windows PowerShell terminal window. The title bar says 'Windows PowerShell'. The prompt is 'PS C:\Users\pralhad>'. The user has entered the command 'cd "C:\Users\pralhad\Desktop\My Script"'. The prompt has changed to 'PS C:\Users\pralhad\Desktop\My Script>'. A blue dashed arrow points from the 'PS' in the first prompt to the explanatory text below. An orange dashed arrow points from the new path in the second prompt to the explanatory text below.

The **PS** at the beginning indicates you're in **PowerShell**.

You are now **"inside"** the folder that contains test.py

Run the Python Script

Type :- `python example.py`



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\pralhad> cd "C:\Users\pralhad\Desktop\My Script"
PS C:\Users\pralhad\Desktop\My Script> python .\example.py
This is running from a relative path!
PS C:\Users\pralhad\Desktop\My Script> |
```

A blue arrow points from the output line "This is running from a relative path!" to a red box labeled "output".



Run a Python Script



Objective:

Learn how to run a Python script from the command line using a **relative path**, and understand why this approach is useful in real-world workflows.



Instructions:

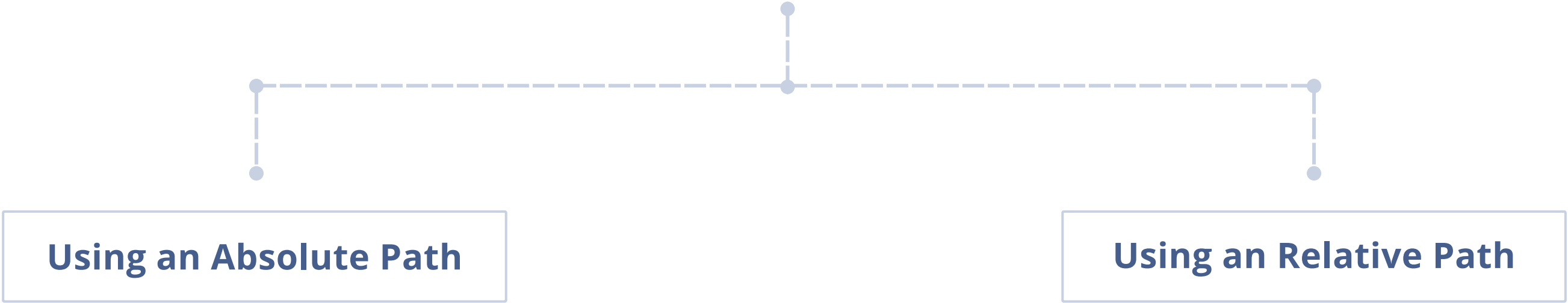
Here's what I want you to do in this task:

- Create a simple Python script with a basic print statement.
- Save the file in a custom folder on your desktop.
- Open PowerShell (or Terminal) and navigate to that folder using the cd command.
- Run the script using just its file name (a relative path).
- Try running it again using the full absolute path inside quotes and observe the difference.

Running Python Scripts in VS Code

- Using an Absolute Path
- Using a Relative Path
- Absolute Path vs. Relative Path

Two Ways to Run a Python Script in VS Code



Using an Absolute Path

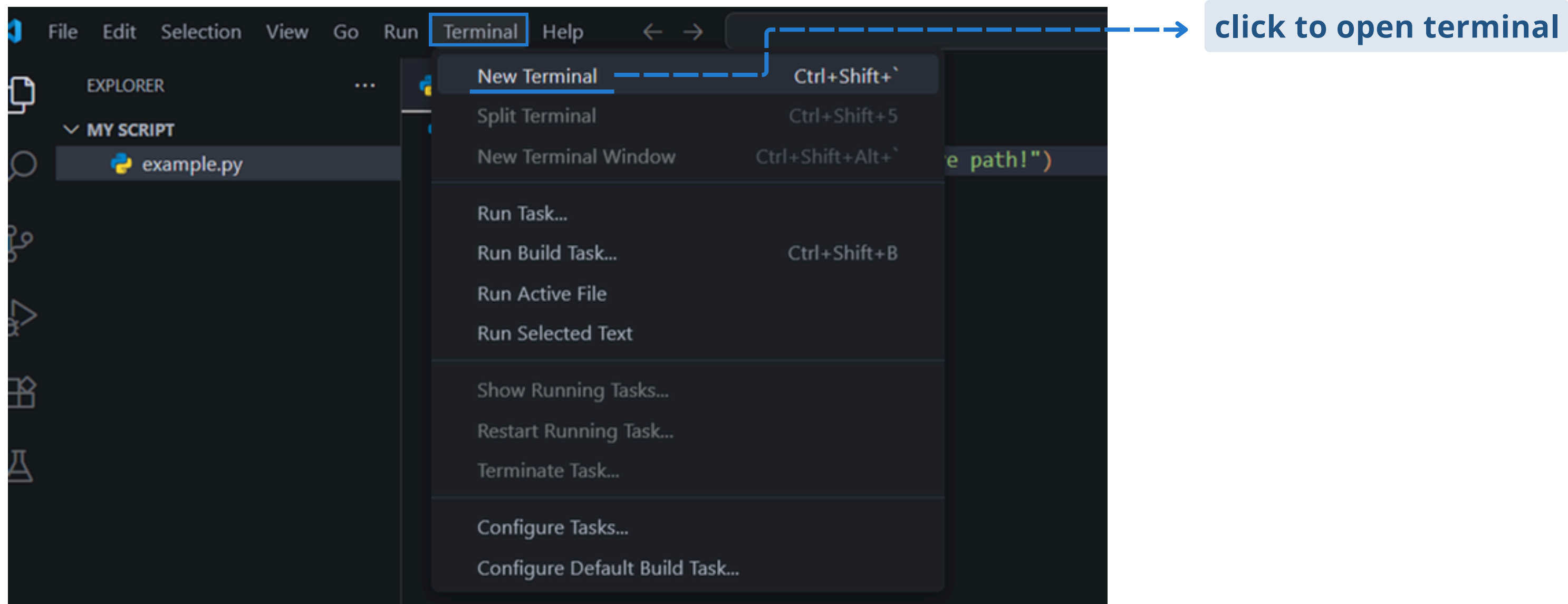
Using an Relative Path

Using an Absolute Path

Let's say we've written a **small script** called **my_script.py**, and the content is simply:

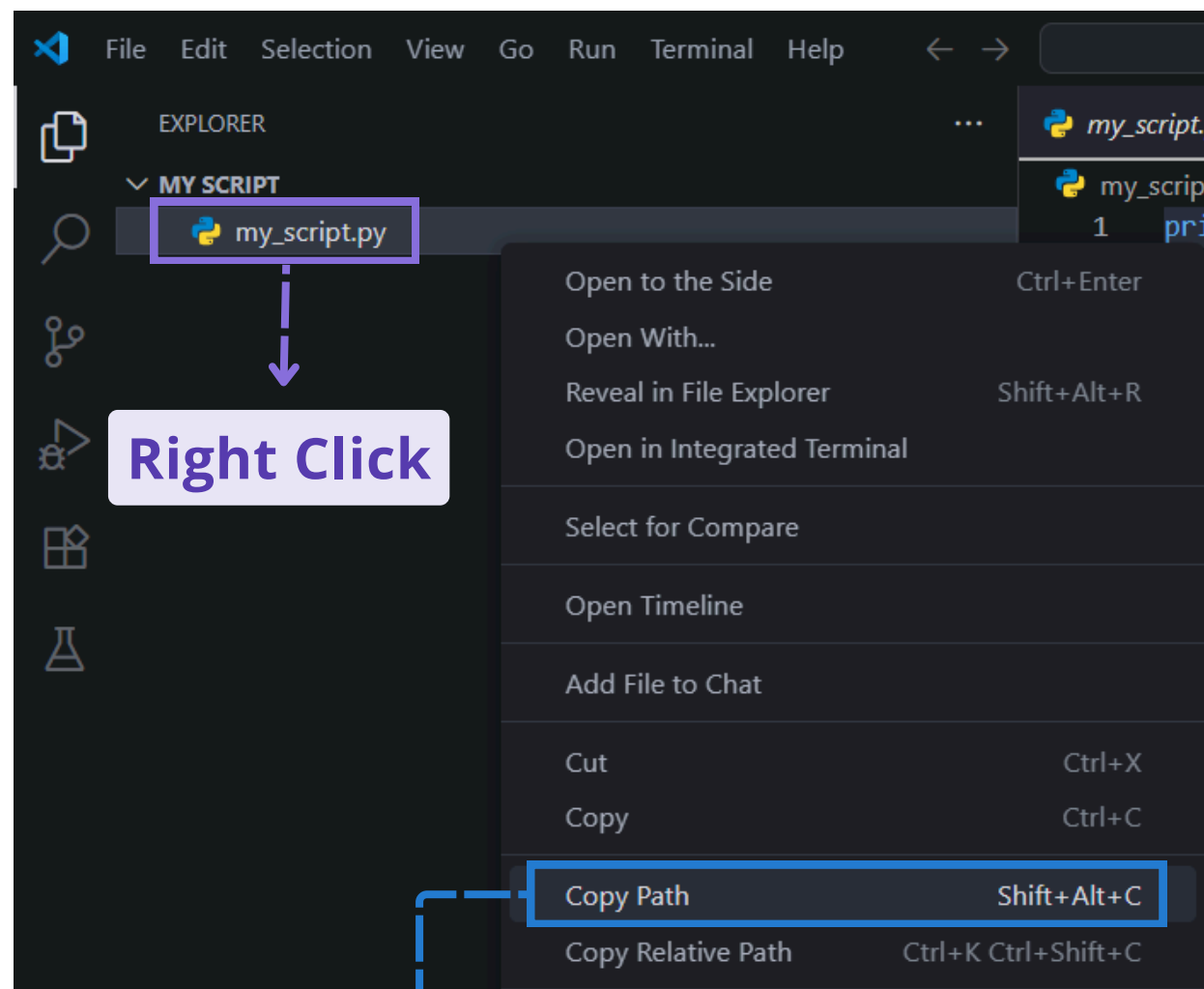
Open a New Terminal in VS Code

- At the top menu, **click Terminal** → **choose New Terminal**.
- This opens the **terminal window** at the **bottom** of your **VS Code workspace**.



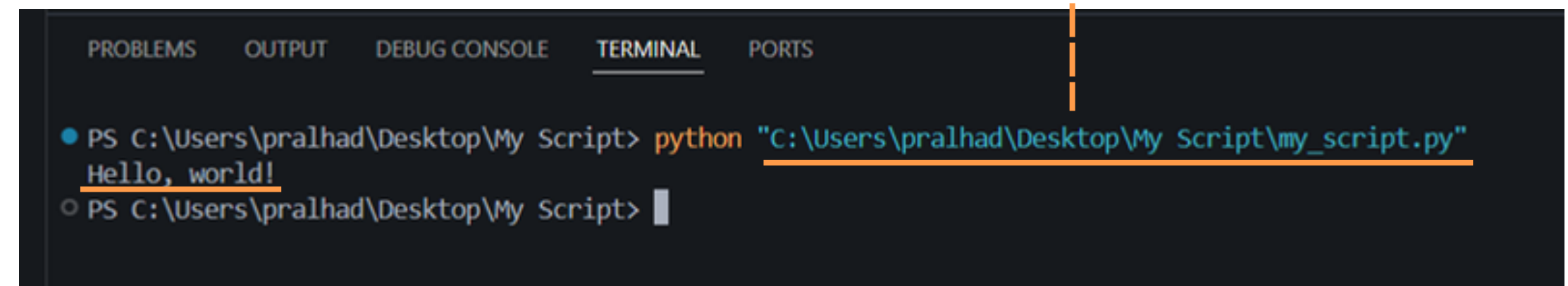
Copy the Absolute Path

- In the Explorer (**left side**), right-click on **my_script.py**.
- Choose “**Copy Path**”.



Click to Copy Absolute Path

Run the Command

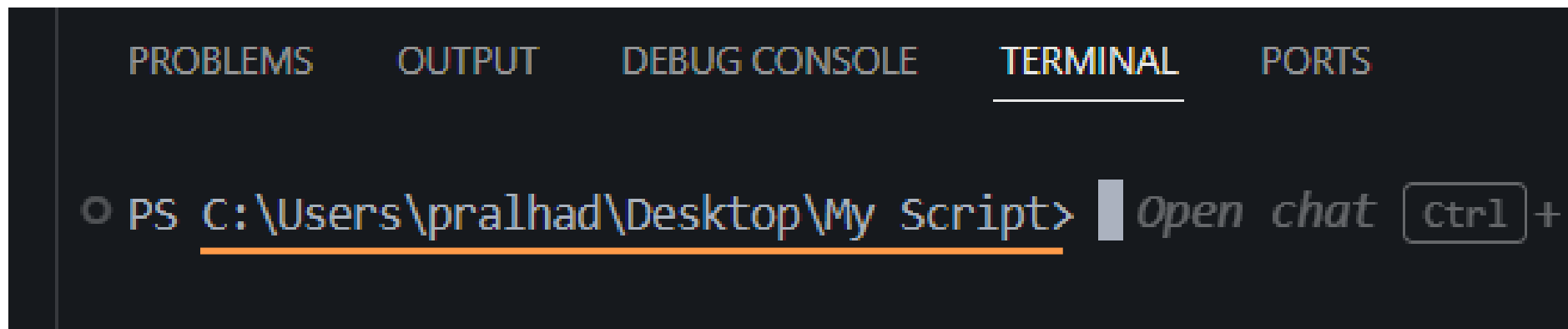


Paste the Path

Using a Relative Path

Now, if your terminal is **already** opened in the **same folder** where your **script** is, there's a **much quicker way** to run it.

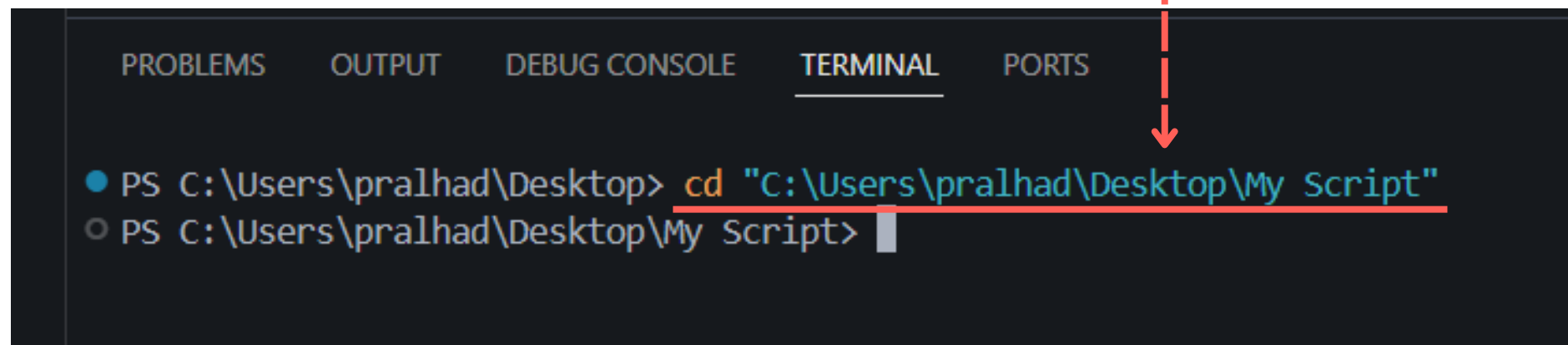
1. Check Your Current Directory



A screenshot of a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. The prompt shows the current directory as C:\Users\pralhad\Desktop\My Script, which is underlined. A cursor is at the end of the line.

```
PS C:\Users\pralhad\Desktop\My Script>
```

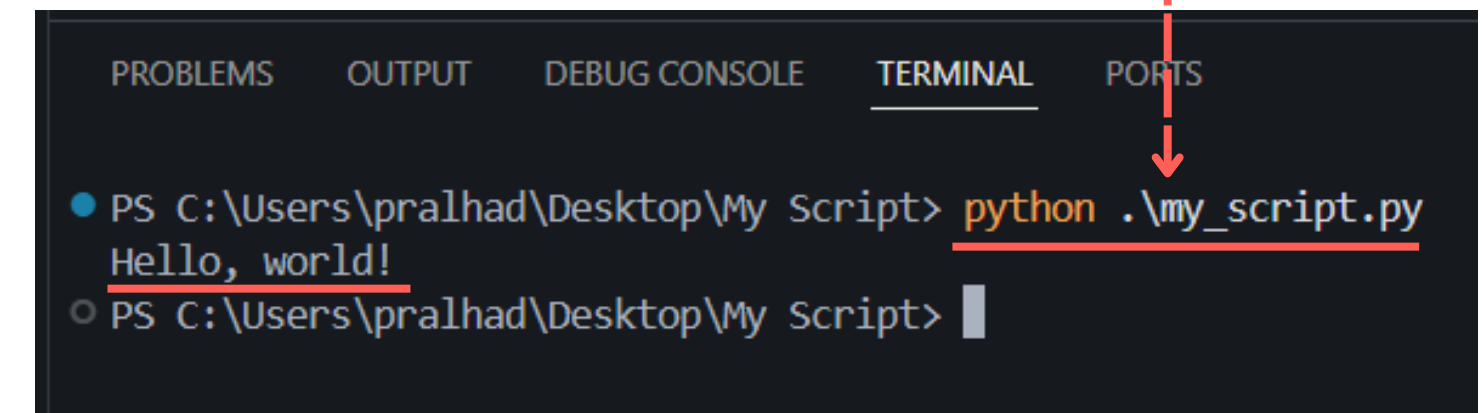
2. Change Directory (If Needed)



A screenshot of a terminal window showing a directory change command. A red dashed arrow points from the title '2. Change Directory (If Needed)' to the `cd` command. The prompt shows the current directory as C:\Users\pralhad\Desktop, and the command `cd "C:\Users\pralhad\Desktop\My Script"` is entered and underlined. The next line shows the prompt has changed to C:\Users\pralhad\Desktop\My Script>.

```
PS C:\Users\pralhad\Desktop> cd "C:\Users\pralhad\Desktop\My Script"
PS C:\Users\pralhad\Desktop\My Script>
```

3. Run the Script



A screenshot of a terminal window showing a script being run. A red dashed arrow points from the title '3. Run the Script' to the `python` command. The prompt shows the current directory as C:\Users\pralhad\Desktop\My Script, and the command `python .\my_script.py` is entered and underlined. The output 'Hello, world!' is shown on the next line. The prompt then returns to C:\Users\pralhad\Desktop\My Script>.

```
PS C:\Users\pralhad\Desktop\My Script> python .\my_script.py
Hello, world!
PS C:\Users\pralhad\Desktop\My Script>
```

Absolute Path vs. Relative Path

Absolute Path (Full Address to the File)

```
python C:\Users\YourName\Documents\my_script.py
```

When to Use:

- You're in a different directory.
- The script is stored deep in the system (nested folders).
- You want to run a file quickly from anywhere.

Benefits:

- Always accurate — no need to know your terminal's current location.
- Ideal for one-off execution or referencing scripts across projects.

Relative Path (Based on Current Directory)

```
cd C:\Users\YourName\Documents  
python my_script.py
```

When to Use:

- You're already in the script's folder (or can easily navigate there).
- Working within a project or pipeline folder structure.

Benefits:

- Cleaner, shorter commands.
 - Great for daily use during development.
 - Teaches good habits around terminal navigation and project structure.
- 

Understanding CLI Tools

- What is a CLI Tool?
- Running a Python File as a CLI Tool
- Running the Script from Its Directory
- Running the Script Without Changing Directories
- Use VS Code's "Run Python File in Terminal"
- When to Use Each Method
- Sample Demo of a Real CLI Tool

What is a CLI Tool?

A **Command-Line Interface (CLI) tool** is a program or script that you **run by typing commands in a terminal**, instead of using a graphical interface (**GUI**).

When you do something like this:

```
python example_script.py
```

You're already using a CLI tool

- **python** is the **tool**,
- **example_script.py** is the **input**.

Examples of CLI Tools File Formats

- **.py** → Python script (run with **python file.py**)
- **.bat** → Windows batch file
- **.sh** → Shell script (**Linux/macOS**)
- **.exe** → Executable program
- **.java** → Compiled Java program

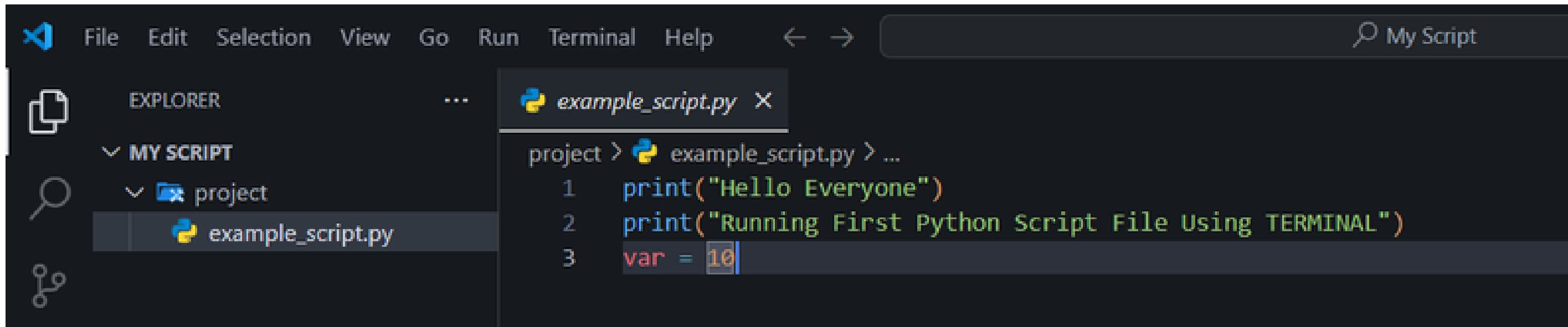
Running a Python File as a CLI Tool

There are **three** main ways to run a **Python script** via the **terminal**. Each has its own use case.

List of Methods

- **Method 1** → Run from the script's directory using **cd**
- **Method 2** → Run using the **full path without changing folders**
- **Method 3** → Use VS Code's built-in "Run Python File" option

File Used: **example_script.py** in a folder called **project**.

A screenshot of the Visual Studio Code (VS Code) interface. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. On the left, the Explorer sidebar shows a project named 'MY SCRIPT' with a subfolder 'project' containing the file 'example_script.py'. The main editor window displays the code for 'example_script.py'. The code consists of three lines: a print statement for 'Hello Everyone', a print statement for 'Running First Python Script File Using TERMINAL', and a variable assignment 'var = 10'. The cursor is positioned at the end of the third line.

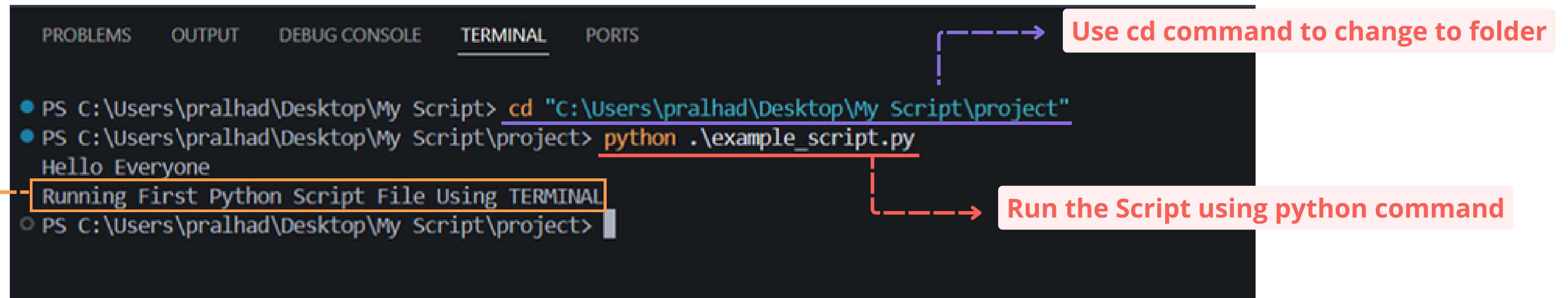
```
project > example_script.py > ...
1  print("Hello Everyone")
2  print("Running First Python Script File Using TERMINAL")
3  var = 10
```

Running the Script from Its Directory

Open Terminal in VS Code

- Use the **shortcut**: Ctrl + backtick (Ctrl + `), or
- Click **Terminal** → New Terminal.

Navigate to the Script Location



The screenshot shows a VS Code terminal window with the following content:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
```

```
• PS C:\Users\pralhad\Desktop\My Script> cd "C:\Users\pralhad\Desktop\My Script\project"
```

```
• PS C:\Users\pralhad\Desktop\My Script\project> python .\example_script.py
```

```
Hello Everyone
```

```
Running First Python Script File Using TERMINAL
```

```
○ PS C:\Users\pralhad\Desktop\My Script\project>
```

Annotations:

- A blue dashed arrow points from the `cd` command to a callout box: "Use cd command to change to folder".
- A red dashed arrow points from the `python` command to a callout box: "Run the Script using python command".
- An orange dashed arrow points from the output "Hello Everyone" to a callout box: "Output".

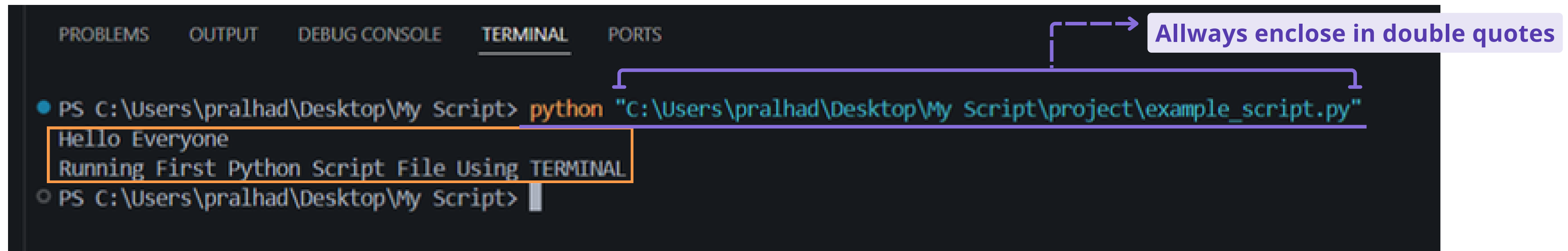
Running the Script Without Changing Directories

This is quicker if you just want to run a file **without navigating** through folders.

1- Copy the Full File Path

- In VS Code, right-click on `example_script.py` → choose **Copy Path**.

2- Run the Script Directly in Terminal



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\pralhad\Desktop\My Script> python "C:\Users\pralhad\Desktop\My Script\project\example_script.py"
Hello Everyone
Running First Python Script File Using TERMINAL
○ PS C:\Users\pralhad\Desktop\My Script>
```

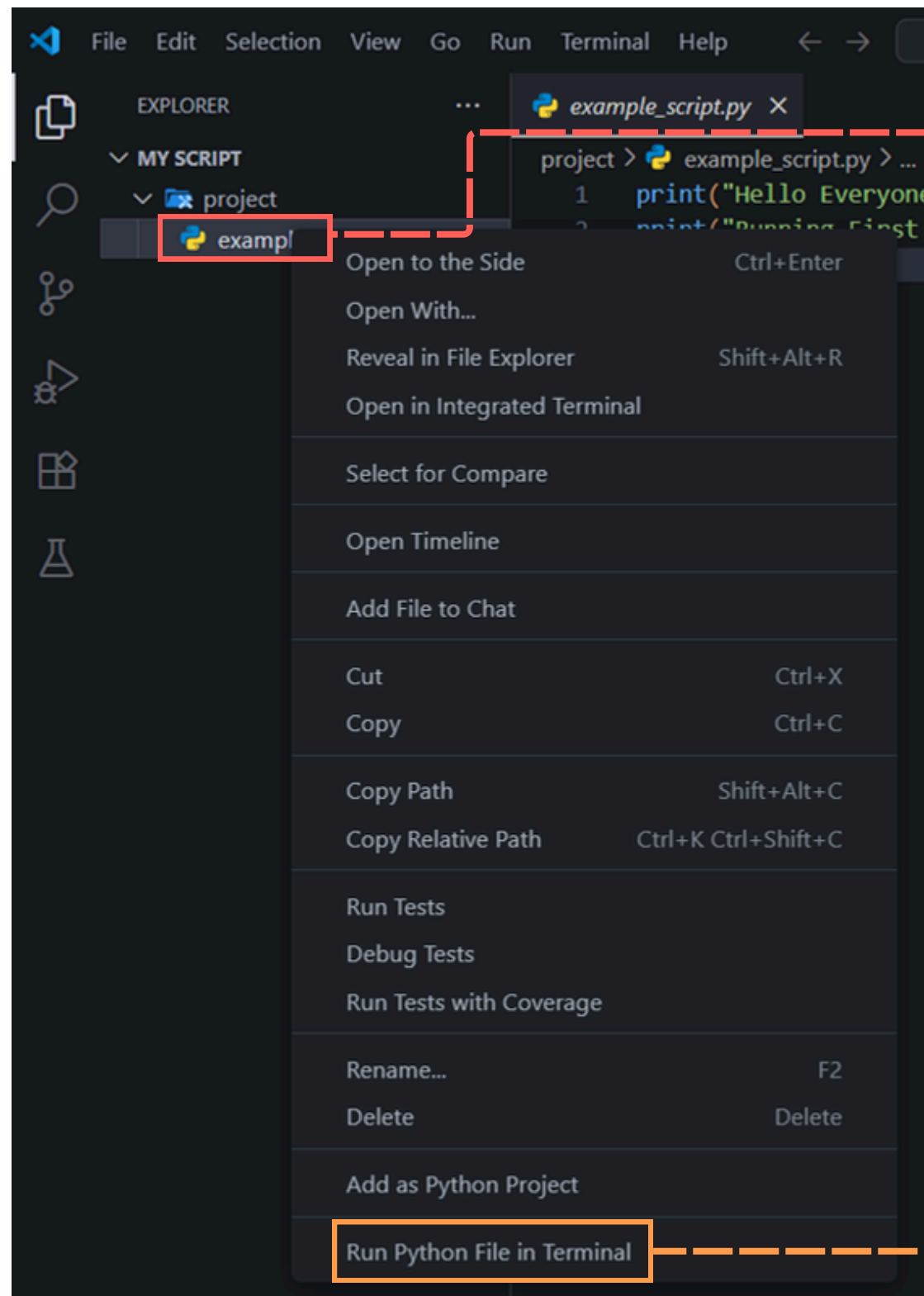
Always enclose in double quotes

Advantages:

- **No** need to change directories, **saving time**.
- Great for **one-off script** execution or testing.

Use VS Code's "Run Python File in Terminal"

In the VS Code Explorer, right-click on **example_script.py**.



Right Click on File

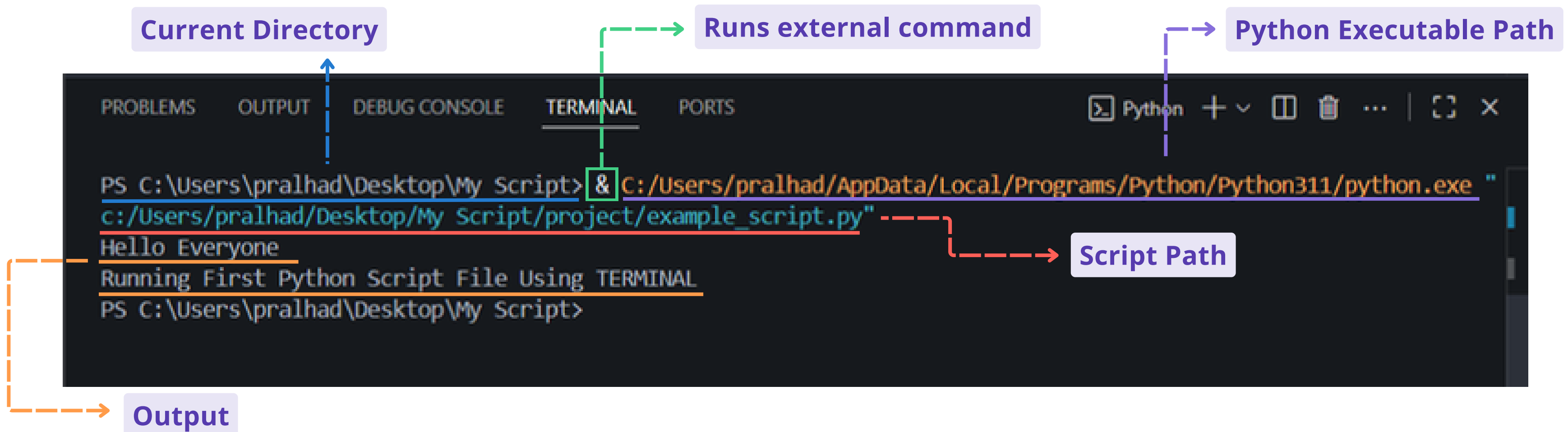
Click to Run Python File

This automatically opens a terminal, navigates to the correct folder, and runs the script.

✓ What VS Code does :

- Opens the terminal (if not open)
- Sets the correct working directory
- Runs the script

- & means run this program
- Used when writing the full path
- Without it, the program won't start



Why Use This?

- Simplifies the process, especially for frequent testing.
- Keeps workflow within the VS Code interface.

When to Use Each Method

Method	Use When...	Why It Matters
• cd + python file.py	→ You're working inside a tool folder with multiple scripts.	• Keeps terminal context focused for repeated use.
• python "full\path\to\file.py"	→ You need to run a script quickly from anywhere.	• Useful for triggering tools from other locations.
• Right-click → Run in Terminal	→ You're actively developing or editing in VS Code.	• Fastest way to test changes during dev.

Sample Demo of a Real CLI Tool

Let's now take it one step further and see how Python scripts can **accept input directly from the command line**.

File: `example_hello_user.py`

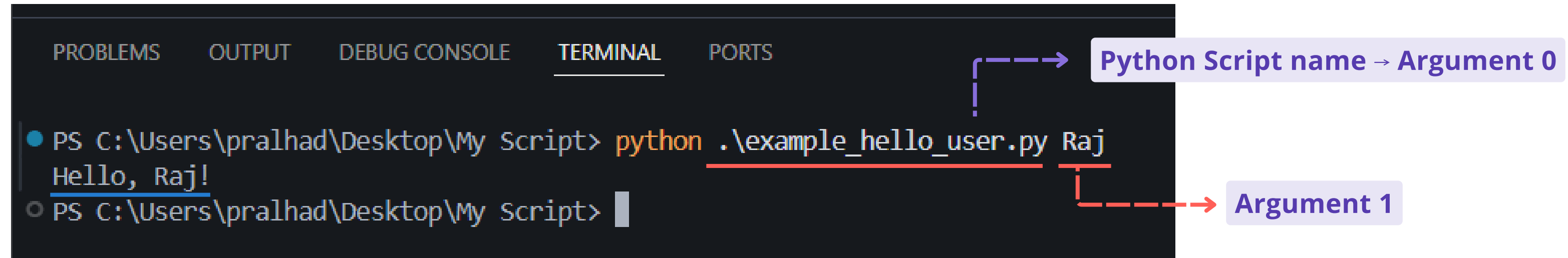
```
import sys
if len(sys.argv) < 2:
    print("Usage: python script.py <your_name>")
    sys.exit(1)
name = sys.argv[1]
print(f"Hello, {name}!")
```

The code is annotated with the following explanations:

- Holds all Arguments passed**: Points to `sys` in the `import sys` statement.
- Check if name was given**: Points to the condition `len(sys.argv) < 2`.
- Show usage if no name provided**: Points to the `print("Usage: python script.py <your_name>")` line.
- Stop the script early**: Points to the `sys.exit(1)` line.
- Get name from command line**: Points to `sys.argv[1]` in the assignment `name = sys.argv[1]`.
- Print the greeting**: Points to the `print(f"Hello, {name}!")` line.

How to Run It

Run with an Argument



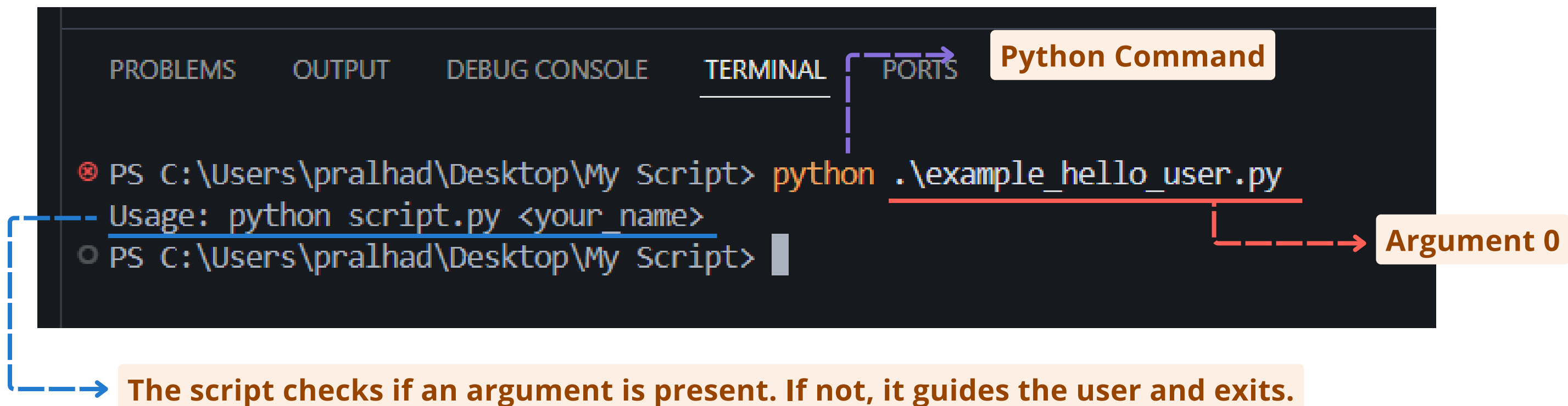
The screenshot shows a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The terminal content is as follows:

```
PS C:\Users\pralhad\Desktop\My Script> python .\example_hello_user.py Raj
Hello, Raj!
```

Annotations:

- A purple dashed line points from the text "Python Script name → Argument 0" to the file path `.\example_hello_user.py`.
- A red dashed line points from the text "Argument 1" to the argument `Raj`.

What If You Don't Provide a Name?



The screenshot shows a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The terminal content is as follows:

```
PS C:\Users\pralhad\Desktop\My Script> python .\example_hello_user.py
Usage: python script.py <your_name>
```

Annotations:

- A purple dashed line points from the text "Python Command" to the `python` command.
- A red dashed line points from the text "Argument 0" to the file path `.\example_hello_user.py`.
- A blue dashed line points from the text "The script checks if an argument is present. If not, it guides the user and exits." to the usage message `Usage: python script.py <your_name>`.

Creating CLI Tools – Using sys.argv

- sys Module
- Understanding sys.argv
- What are Command-Line Arguments?
- Syntax for Passing Arguments

sys Module

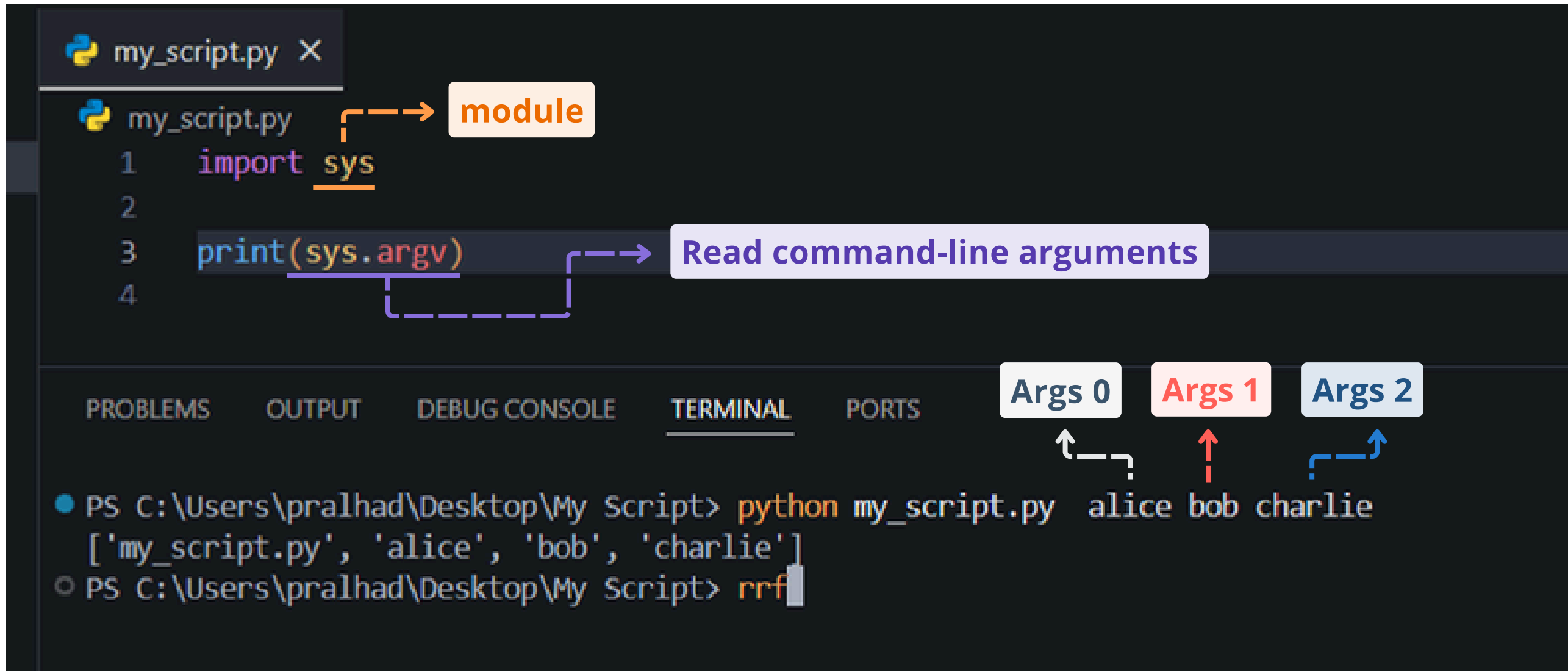
- The **sys** module comes **built-in with Python** — no installation needed.
- It allows your **script** to interact with the **Python runtime system**.

What we Can Do With sys

Function	Use Case
• sys.argv	→ Read command-line arguments
• sys.exit()	→ Exit the script early
• sys.version	→ Check the version of Python
• sys.path	→ Modify module search paths

To use it, we just import it: `import sys`

Understanding sys.argv



The screenshot shows a code editor with a file named `my_script.py`. The code is as follows:

```
1 import sys
2
3 print(sys.argv)
4
```

Annotations in the image:

- An orange box labeled **module** points to the `sys` module in the `import` statement.
- A purple box labeled **Read command-line arguments** points to the `sys.argv` argument in the `print` statement.

Below the code editor is a terminal window with the following output:

```
PS C:\Users\pralhad\Desktop\My Script> python my_script.py alice bob charlie
['my_script.py', 'alice', 'bob', 'charlie']
PS C:\Users\pralhad\Desktop\My Script> rrf
```

Annotations in the terminal:

- Three boxes labeled **Args 0**, **Args 1**, and **Args 2** are positioned above the terminal output.
- Arrows point from these boxes to the corresponding elements in the list: `['my_script.py', 'alice', 'bob', 'charlie']`. Specifically, **Args 0** points to `'my_script.py'`, **Args 1** points to `'alice'`, and **Args 2** points to `'bob'`.

Each Arguments are passed **separated by Spaces**

✓ **Remember:** All arguments are strings, even if they look like numbers.

What are Command-Line Arguments?

Command-line arguments are **values you pass to a script when running it from the terminal.**

They allow you to:

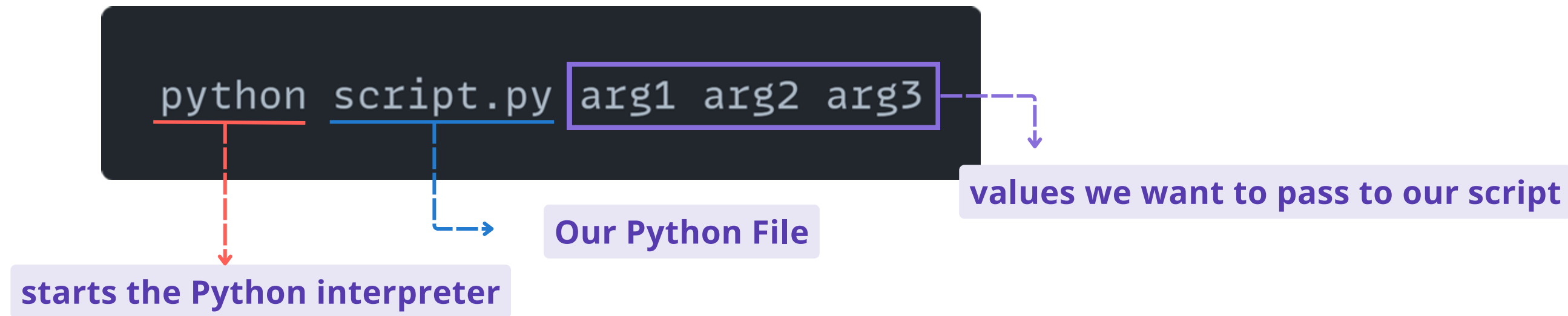
- Dynamically change what your script does
- Avoid hardcoding values
- Provide input without modifying the code

Useful when automating tasks, building tools, or running scripts in pipelines.

Syntax for Passing Arguments

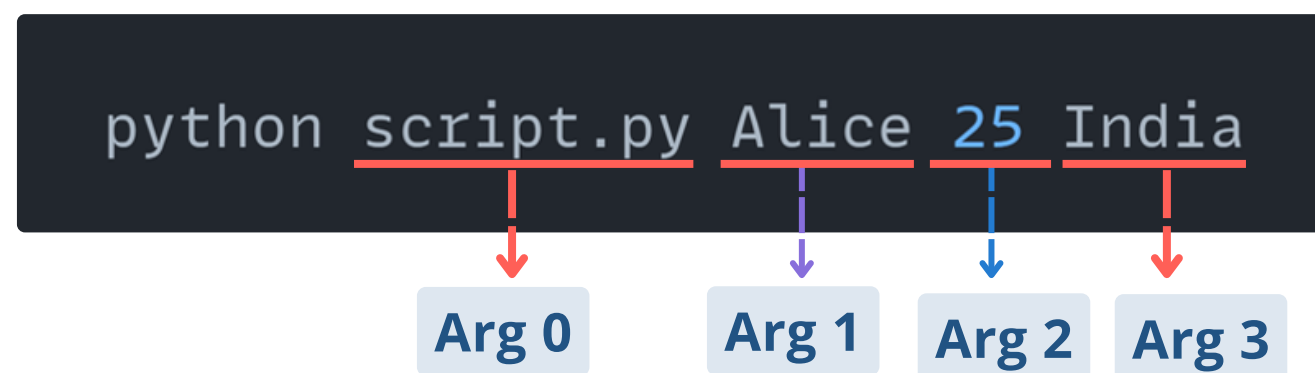
- **Command-line arguments** are values we pass when running a **Python script** from the terminal.
- These arguments are passed after the **script name**, separated by **spaces**.

The syntax for passing arguments:



How Do We Use Them ?

We simply type the arguments after the **script name**, separated by **spaces**:



Using Command Line Arguments

- Passing One Argument
- Passing Multiple Arguments

Passing One Arguments

Let's say you have the following **script** named **script.py**:

```
import sys  
  
name = sys.argv[1]  
print(f"Hello, {name}!")
```

name = sys.argv[1]

Holds the List of Args pass

Takes 1 Argument

Open Terminal → Run the Command

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
● PS C:\Users\pralhad\Desktop\My Script> python .\example_hello_user.py Raj  
Hello, Raj!  
○ PS C:\Users\pralhad\Desktop\My Script> 
```

Arg 0

Arg 1

Passing Multiple Arguments

Let's now modify the script to accept three inputs — first name, last name, and age.

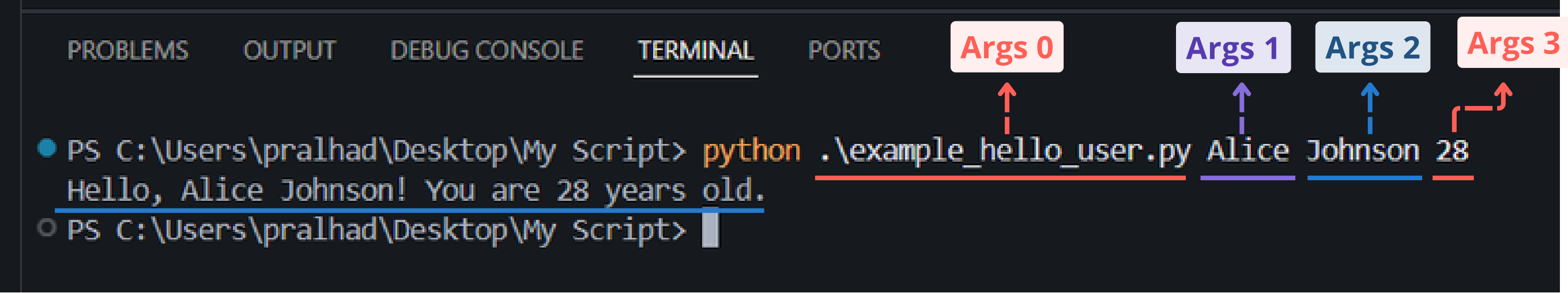
```
import sys

first_name = sys.argv[1]
last_name = sys.argv[2]
age = sys.argv[3]

print(f"Hello, {first_name} {last_name}! You are {age} years old.")
```



Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
```

● PS C:\Users\pralhad\Desktop\My Script> python .\example_hello_user.py Alice Johnson 28
Hello, Alice Johnson! You are 28 years old.

○ PS C:\Users\pralhad\Desktop\My Script> █

Working with Arguments

- Checking All Passed Arguments
- Accessing Individual Arguments
- Checking the Type of `sys.argv`

Checking All Passed Arguments

We can view everything passed to our script (including the script name itself) by printing the entire **sys.argv** list.

The image shows a code editor window with a file named `my_script.py`. The code contains the following lines:

```
1 import sys
2
3 print(sys.argv)
4
```

Annotations in the code editor include:

- An orange box labeled **module** with an arrow pointing to the `sys` import on line 1.
- A purple box labeled **Read command-line arguments** with an arrow pointing to the `sys.argv` argument in the `print` statement on line 3.

Below the code editor is a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, **TERMINAL**, and PORTS. The terminal shows the following commands and output:

```
PS C:\Users\pralhad\Desktop\My Script> python my_script.py alice bob charlie
['my_script.py', 'alice', 'bob', 'charlie']
PS C:\Users\pralhad\Desktop\My Script> rrf
```

Annotations in the terminal include:

- Three boxes labeled **Args 0**, **Args 1**, and **Args 2** are positioned above the terminal output.
- Arrows point from these boxes to the corresponding elements in the output list: `'my_script.py'` (Args 0), `'alice'` (Args 1), and `'bob'` (Args 2).

Accessing Individual Arguments

You can access each argument by its **index**:

```
my_script.py X
my_script.py
1 import sys
2
3 print("Script name:", sys.argv[0])
4 print("First argument:", sys.argv[1])
5 print("Second argument:", sys.argv[2])
```

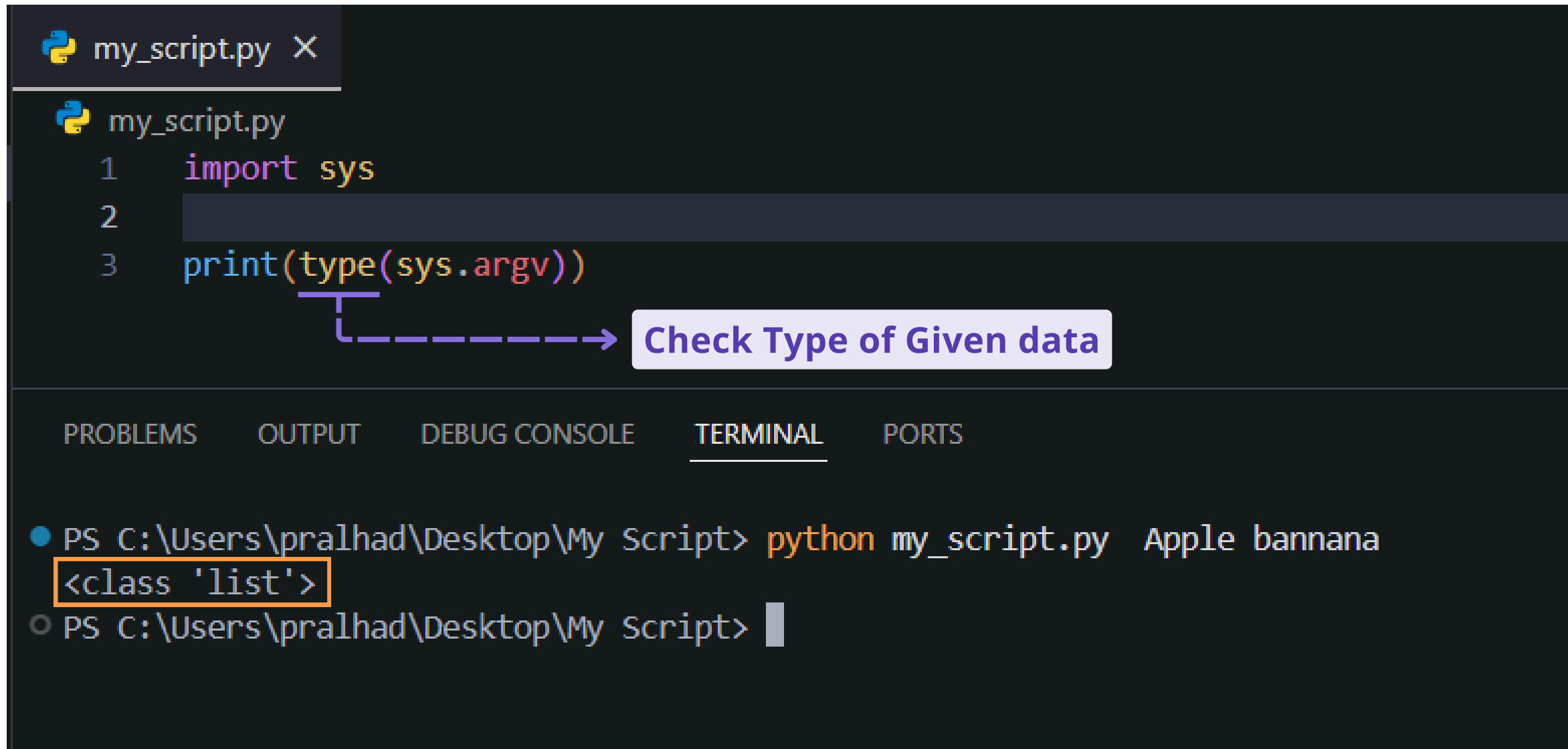
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\pralhad\Desktop\My Script> python my_script.py Apple bannana
Script name: my_script.py
First argument: Apple
Second argument: bannana
PS C:\Users\pralhad\Desktop\My Script>
```

⚠ Indexing starts from **0**, and all values are strings.

Checking the Type of sys.argv

To confirm that **sys.argv** is a list:



The screenshot shows a code editor with a file named `my_script.py`. The code contains three lines: `import sys`, a blank line, and `print(type(sys.argv))`. A dashed line points from the `sys.argv` in the `print` statement to a callout box that says "Check Type of Given data". Below the code editor is a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The terminal shows the command `python my_script.py Apple bannana` being executed, and the output `<class 'list'>` is displayed and highlighted with an orange box. The prompt `PS C:\Users\pralhad\Desktop\My Script>` is visible on both lines of the terminal.

```
my_script.py X
my_script.py
1 import sys
2
3 print(type(sys.argv))
           |-----> Check Type of Given data

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\pralhad\Desktop\My Script> python my_script.py Apple bannana
  <class 'list'>
○ PS C:\Users\pralhad\Desktop\My Script>
```

Practical CLI Tool Example

- A Problem Statement
- Creating a Log Analyzer Tool
- Show Count of Matches
- Why This Tool is Useful in VFX

A Problem Statement

Scenario:

In a VFX studio, systems generate large **log files** — often thousands of lines — that track everything from startup processes to crashes.

Imagine a file like this:

```
2024-07-23 09:00:00 DEBUG: Debugging the application startup process.
2024-07-23 09:05:00 INFO: Application started successfully.
2024-07-23 09:10:00 WARNING: Low memory warning. Available memory is below the threshold.
2024-07-23 09:15:00 ERROR: FileNotFoundException: Unable to locate the configuration file.
2024-07-23 09:20:00 ERROR: OutOfMemory: System ran out of memory while processing the data.
2024-07-23 09:25:00 INFO: Memory cleanup process initiated.
2024-07-23 09:30:00 INFO: Memory cleanup completed successfully.
2024-07-23 09:35:00 ERROR: NullReference: Attempted to access an object that is null.
2024-07-23 09:40:00 CRITICAL: SystemCrash: The application has encountered a critical error and needs to close.
2024-07-23 09:45:00 INFO: Attempting to restart the application.
2024-07-23 09:50:00 DEBUG: Restart process initiated.
2024-07-23 09:55:00 INFO: Application restarted successfully.
2024-07-23 10:00:00 WARNING: High CPU usage detected. Monitoring the performance.
2024-07-23 10:05:00 DEBUG: Analyzing the CPU usage trends.
2024-07-23 10:10:00 ERROR: IOException: Error occurred while reading from the disk.
2024-07-23 10:15:00 INFO: Disk read operation retried successfully.
2024-07-23 10:20:00 DEBUG: Debugging network connection issues.
2024-07-23 10:25:00 ERROR: NetworkTimeout: Connection to the server timed out.
2024-07-23 10:30:00 INFO: Network connection re-established.
2024-07-23 10:35:00 DEBUG: Running diagnostics on network stability.
2024-07-23 10:40:00 INFO: Diagnostics completed with no significant issues found.
2024-07-23 10:45:00 CRITICAL: SecurityBreach: Unauthorized access attempt detected.
2024-07-23 10:50:00 INFO: Security protocols engaged and access blocked.
```


The Problem

- Imagine you're a **Technical Director (TD)** or pipeline support person in a VFX studio.
- One day, your **supervisor walks** over and says:

Task 1:

“Can you give me a list of all **ERROR** messages from this log.txt file, along with how many errors there are?”

Task 2 (a few minutes later):

“Now I also need a list of all **WARNING** messages — and their count too.”

- At first glance, this sounds easy — especially if the **log file is small**.
- But in **real-world VFX pipelines**, log files often include:
 - Thousands of lines of output
 - Logs from render nodes, software crashes, simulations
 - Continuous data from days or weeks of production

We're going to create a Python script that:

- Takes the **log file** as an **input**.
 - Filters lines based on a **specific keyword** (like **ERROR** or **WARNING**).
 - Displays the **filtered lines**.
- 

Creating a Log Analyzer Tool

```
import sys
```

```
first_arg = sys.argv[1]
```

Takes Log File as First Argument = log.txt

```
with open(first_arg, 'r') as f:  
    data = f.readlines()
```

Read log file

```
second_arg = sys.argv[2]
```

```
if second_arg == "INFO":
```

```
    for each_line in data:
```

```
        if "INFO" in each_line.strip():  
            print(each_line.strip())
```

Loop Through Logs and Filter it by INFO Keyword

```
if second_arg == "ERROR":
```

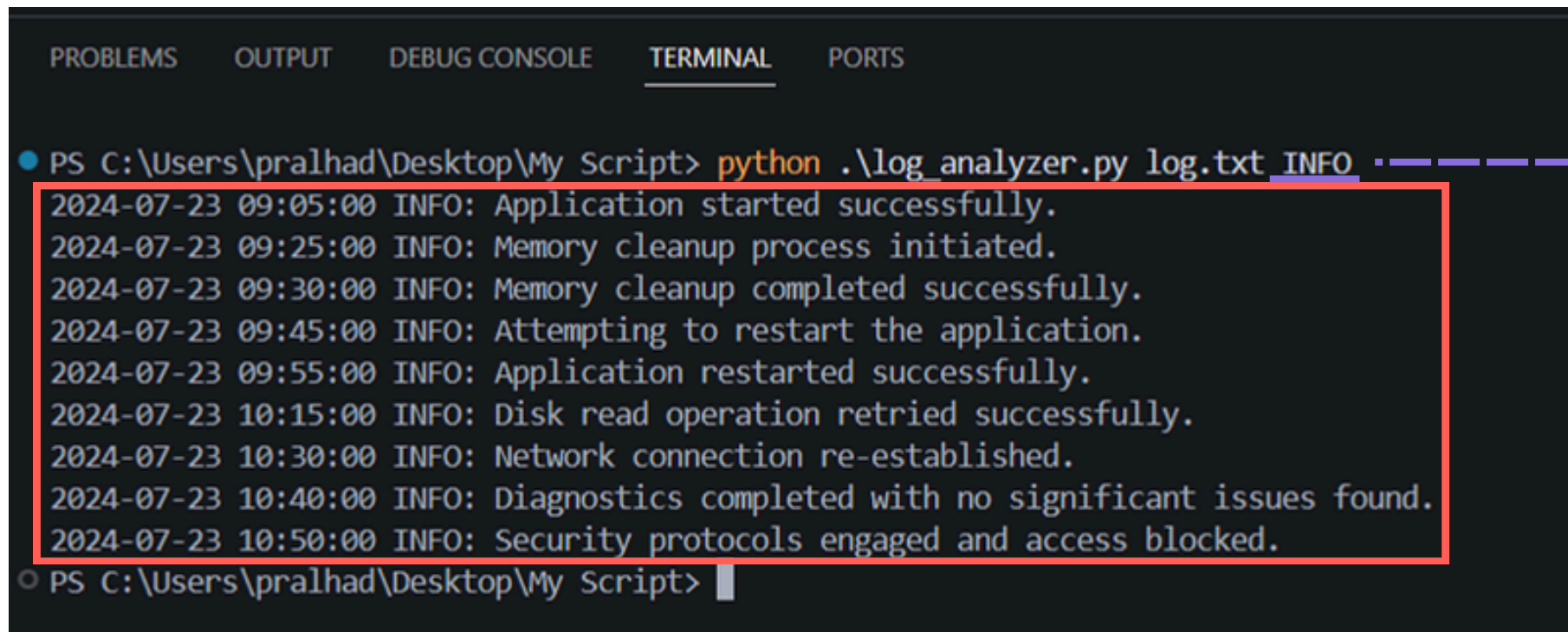
```
    for each_line in data:
```

```
        if "ERROR" in each_line.strip():  
            print(each_line.strip())
```

Loop Through Logs and Filter it by ERROR Keyword

Running the Tool

Save the script and run it from the terminal. Use the following command:



The screenshot shows a terminal window with the following content:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
```

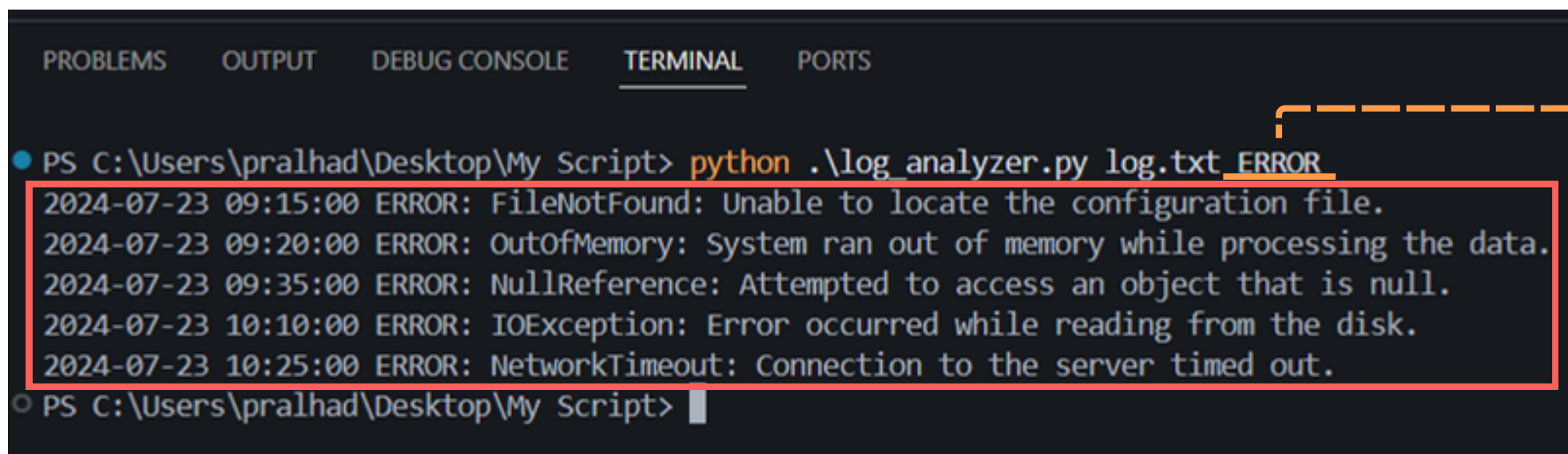
● PS C:\Users\pralhad\Desktop\My Script> python .\log_analyzer.py log.txt INFO

```
2024-07-23 09:05:00 INFO: Application started successfully.
2024-07-23 09:25:00 INFO: Memory cleanup process initiated.
2024-07-23 09:30:00 INFO: Memory cleanup completed successfully.
2024-07-23 09:45:00 INFO: Attempting to restart the application.
2024-07-23 09:55:00 INFO: Application restarted successfully.
2024-07-23 10:15:00 INFO: Disk read operation retried successfully.
2024-07-23 10:30:00 INFO: Network connection re-established.
2024-07-23 10:40:00 INFO: Diagnostics completed with no significant issues found.
2024-07-23 10:50:00 INFO: Security protocols engaged and access blocked.
```

○ PS C:\Users\pralhad\Desktop\My Script>

A dashed blue arrow points from the `INFO` filter in the command to a callout box.

Filter all INFO Logs



The screenshot shows a terminal window with the following content:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
```

● PS C:\Users\pralhad\Desktop\My Script> python .\log_analyzer.py log.txt ERROR

```
2024-07-23 09:15:00 ERROR: FileNotFoundError: Unable to locate the configuration file.
2024-07-23 09:20:00 ERROR: OutOfMemory: System ran out of memory while processing the data.
2024-07-23 09:35:00 ERROR: NullReference: Attempted to access an object that is null.
2024-07-23 10:10:00 ERROR: IOException: Error occurred while reading from the disk.
2024-07-23 10:25:00 ERROR: NetworkTimeout: Connection to the server timed out.
```

○ PS C:\Users\pralhad\Desktop\My Script>

A dashed orange arrow points from the `ERROR` filter in the command to a callout box.

Filter all ERROR Logs

Show Count of Matches

- Right now, our tool prints filtered lines from a log file (like **ERROR** or **WARNING**).
- Let's now improve it by also showing how many matching entries were found.

```
import sys

log_file = sys.argv[1]

with open(log_file, 'r') as file:
    log_lines = file.readlines()

keyword = sys.argv[2]
count = 0

for line in log_lines:
    if keyword in line:
        print(line.strip())
        count += 1

print(f"\nTotal {keyword} entries: {count}")
```

To keep track of how many matches we find

Loop & Filter logs based on given keyword

Now, if you run the tool with the same command:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\pralhad\Desktop\My Script> python .\log_analyzer.py log.txt ERROR
2024-07-23 09:15:00 ERROR: FileNotFoundError: Unable to locate the configuration file.
2024-07-23 09:20:00 ERROR: OutOfMemory: System ran out of memory while processing the data.
2024-07-23 09:35:00 ERROR: NullReference: Attempted to access an object that is null.
2024-07-23 10:10:00 ERROR: IOException: Error occurred while reading from the disk.
2024-07-23 10:25:00 ERROR: NetworkTimeout: Connection to the server timed out.

Total ERROR entries: 5

PS C:\Users\pralhad\Desktop\My Script>
```

Filtering all ERROR

Why This Tool is Useful in VFX

Tools like this are extremely common in VFX production:

- **Debugging Pipelines:** When something breaks, you can quickly identify **errors** and **warnings** in the **logs**.
- **Reporting:** Supervisors may ask for a **summary of issues** or **specific types of logs**.
- **Automation:** Instead of **manually scrolling** through logs, you can **instantly filter** and count what you **need**.

This is a simple example, but in real-world scenarios, tools like this are extended further to:

- **Save filtered** results to a file.
- Handle **multiple keywords** at once (e.g., both **ERROR** and **WARNING**).
- Integrate with **larger pipeline systems**.

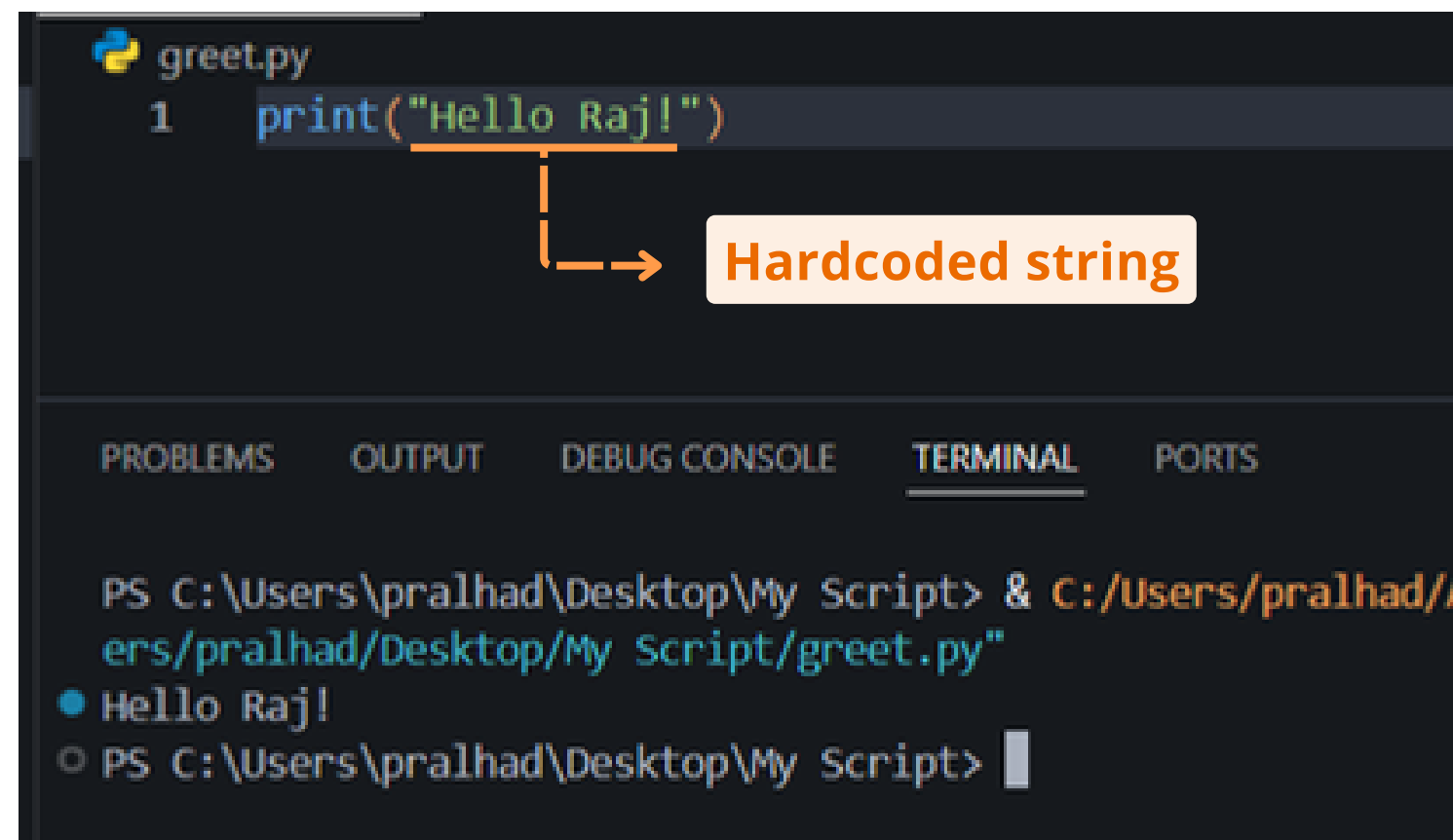
Why Use CLI Arguments?

- Dynamic Input Without Hardcoding
- Reusability for Different Scenarios
- Control Logic Based on Input
- Automating Repetitive Tasks

Dynamic Input Without Hardcoding

Scenario: You're writing a greeting tool where users can enter their name.

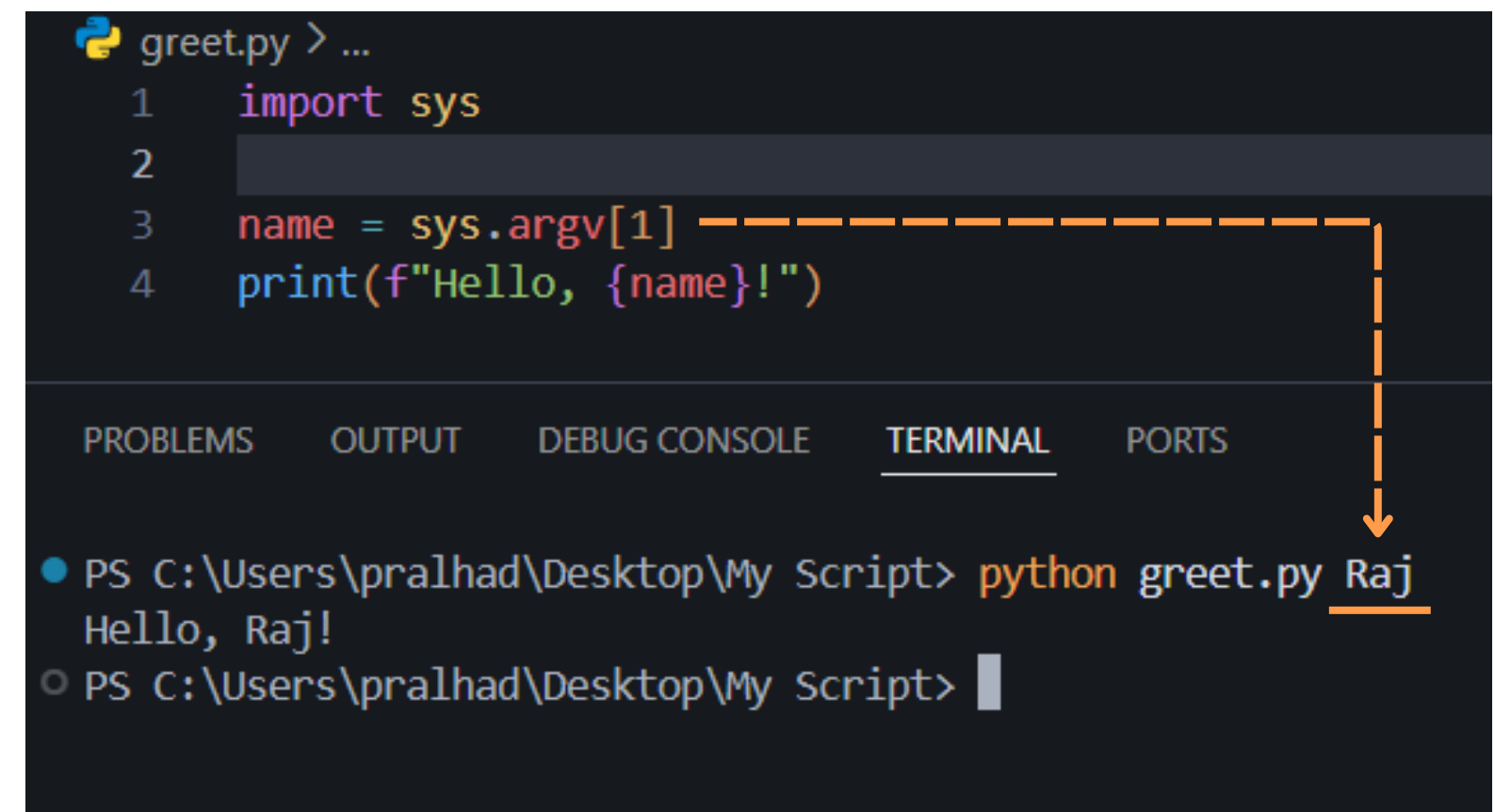
✗ Hardcoded script:



The screenshot shows a code editor with a file named `greet.py`. Line 1 contains the code `print("Hello Raj!")`. An orange arrow points from the string `"Hello Raj!"` to a yellow box labeled **Hardcoded string**. Below the editor is a terminal window showing the command `PS C:\Users\pralhad\Desktop\My Script> & C:/Users/pralhad/Desktop/My Script/greet.py` and its output `• Hello Raj!`.

- Every time we want to greet someone else,
- We have to edit the file.

✓ CLI version:



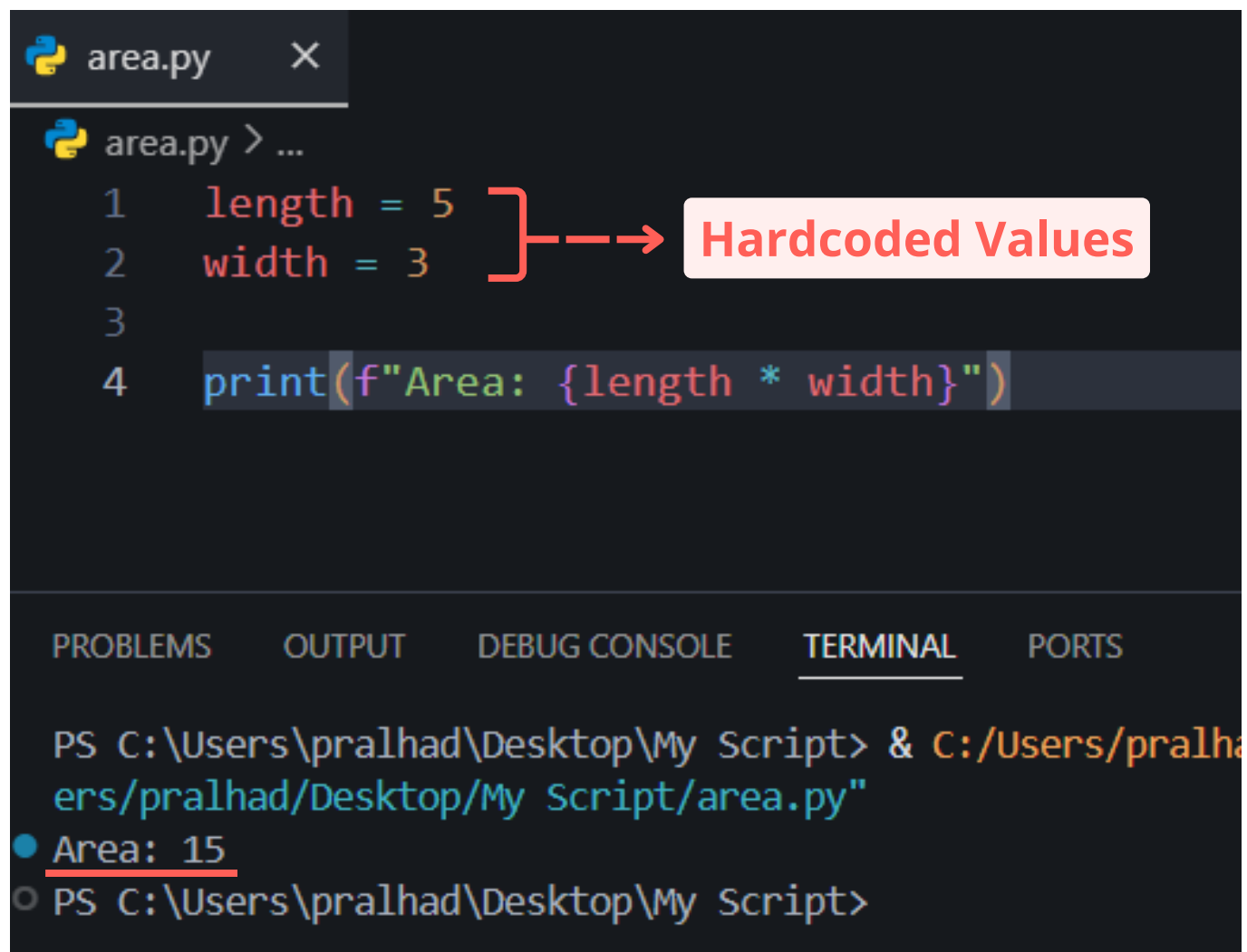
The screenshot shows a code editor with a file named `greet.py`. Line 1 contains `import sys`. Line 3 contains `name = sys.argv[1]`. Line 4 contains `print(f"Hello, {name}!")`. An orange dashed arrow points from `sys.argv[1]` to the word `Raj` in the terminal command `python greet.py Raj`. The terminal output shows `• PS C:\Users\pralhad\Desktop\My Script> python greet.py Raj` followed by `Hello, Raj!`.

- No need to change the script.
- Just pass a different name as an argument.

Reusability for Different Scenarios

Scenario: You need to calculate the area of various rectangles — length and width change often.

✗ Hardcoded dimensions:



The screenshot shows a code editor with a file named `area.py`. The code is as follows:

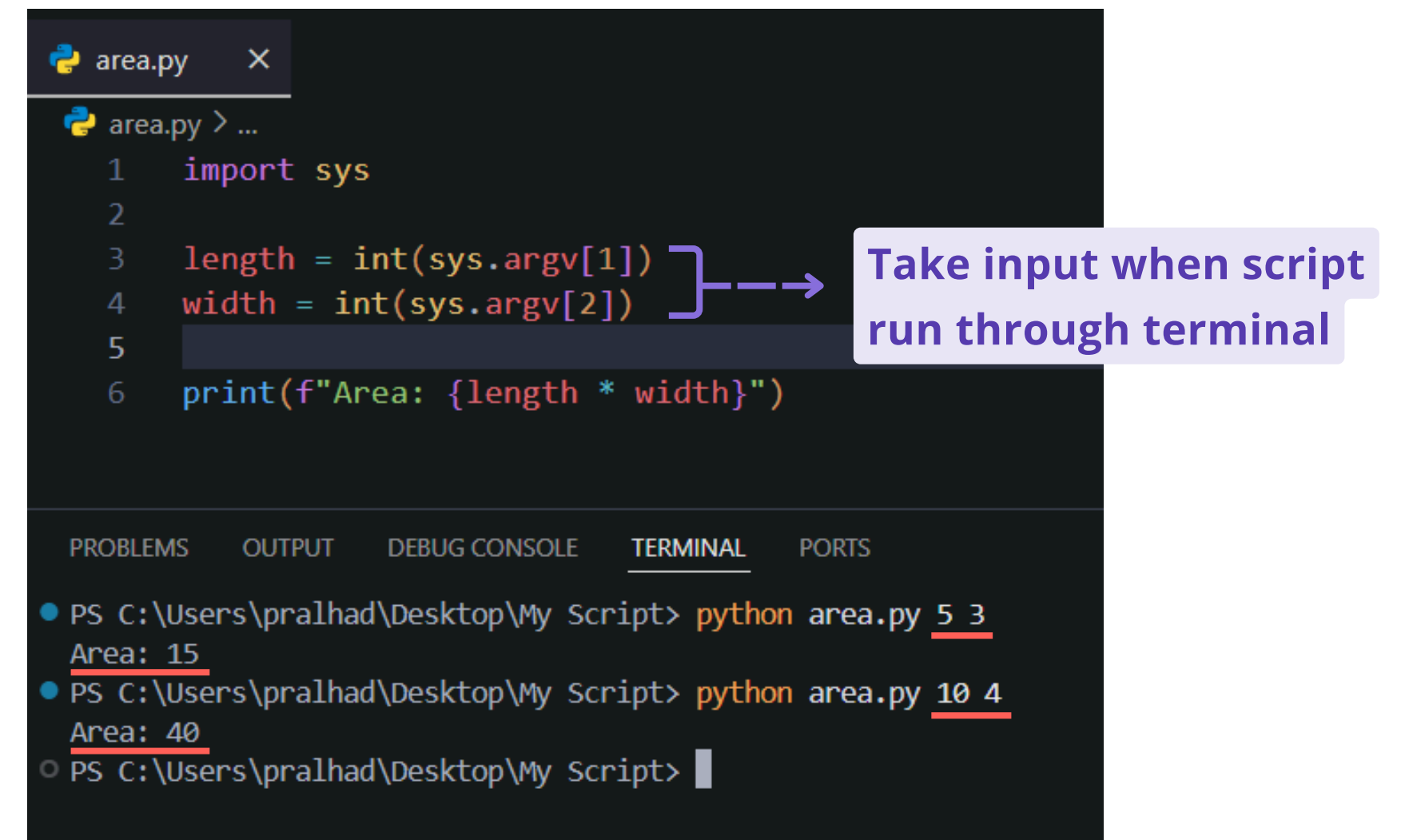
```
1 length = 5
2 width = 3
3
4 print(f"Area: {length * width}")
```

A red bracket on the right side of lines 1 and 2 points to a text box labeled "Hardcoded Values".

The terminal output shows the command `PS C:\Users\pralhad\Desktop\My Script> & C:/Users/pralhad/Desktop/My Script/area.py` and the output `Area: 15`.

Always calculates the same area.

✓ CLI version:



The screenshot shows a code editor with a file named `area.py`. The code is as follows:

```
1 import sys
2
3 length = int(sys.argv[1])
4 width = int(sys.argv[2])
5
6 print(f"Area: {length * width}")
```

A purple bracket on the right side of lines 3 and 4 points to a text box labeled "Take input when script run through terminal".

The terminal output shows the command `PS C:\Users\pralhad\Desktop\My Script> python area.py 5 3` and the output `Area: 15`. The next command is `PS C:\Users\pralhad\Desktop\My Script> python area.py 10 4` and the output is `Area: 40`.

Same script works for any size

Control Logic Based on Input

Scenario: You're building a simple calculator that either adds or multiplies two numbers.

✗ Hardcoded Logic

```
calculator.py X
calculator.py > ...
1 operation = "add"
2 num1 = 5
3 num2 = 3
4
5 if operation == "add":
6     print(f"Sum: {num1 + num2}")
7 elif operation == "multiply":
8     print(f"Product: {num1 * num2}")
9 else:
10    print("Invalid operation")
11
12 # Output: Sum: 8
13
```

- We have to edit the code every time we want to:
 - Change the **operation value** (add → multiply)
 - Change the numbers

✓ CLI version:

```
calculator.py X
calculator.py > ...
1 import sys
2
3 operation = sys.argv[1]
4 num1 = int(sys.argv[2])
5 num2 = int(sys.argv[3])
6
7 if operation == "add":
8     print(f"Sum: {num1 + num2}")
9 elif operation == "multiply":
10    print(f"Product: {num1 * num2}")
11 else:
12    print("Invalid operation")
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\pralhad\Desktop\My Script> python calculator.py add 5 3
Sum: 8
PS C:\Users\pralhad\Desktop\My Script> python calculator.py add 20 45
Sum: 65
PS C:\Users\pralhad\Desktop\My Script>
```

- ✓ Choose the action at runtime.
- ✓ One script handles multiple operations.

Automating Repetitive Tasks

Scenario: Imagine processing 10 different files in a folder.

✗ Hardcoded

```
file_processor.py X
file_processor.py > ...
1  file_path = "/path/to/file1.txt"
2
3  print(f"Processing file: {file_path}")
4
5  # Output: Processing file: /path/to/file1.txt
```

Hardcoded path

We Need to Edit Script for Every New File

✓ CLI version:

```
file_processor.py X
file_processor.py > ...
1  import sys
2
3  file_path = sys.argv[1]
4
5  print(f"Processing file: {file_path}")
```

Provide path during run

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\pralhad\Desktop\My Script> python file_processor.py /files/data1.txt
Processing file: /files/data1.txt
PS C:\Users\pralhad\Desktop\My Script> python file_processor.py /files/test.py
Processing file: /files/test.py
PS C:\Users\pralhad\Desktop\My Script> |
```

- Run the same script on any file
- Ideal for batch processing and automation

Difference Between Hardcoding & CLI Arguments

Hardcoding = Change code every time

CLI Argument = Change input at runtime



Walking to TV every time
to change channels



Using a remote –
quick and flexible

Benefits Summary

Benefit

- **Saves Time**
- **Gives Control**
- **Increases Reusability**
- **Makes Automation Possible**

What It Means

- No need to open and edit scripts for each run
 - You decide inputs or actions from the terminal
 - One script works for many use cases
 - Easy to plug into pipelines, batch jobs, or external tools
- 

CLI Tools in the VFX Industry

- Where Are CLI Tools Used?
- Submitting a Render Job
- Why Do We Use CLI for Render Submission?
- How Do Artists Use CLI Tools Without Typing Commands?
- Cleaning Up a Project Clutters
- Why Use CLI Tools for Automation?

Submitting a Render Job

- Let's say you've got a shot that needs rendering — **maybe 100 frames** from **Maya**.
- Instead of manually launching Maya and adjusting settings for each shot, a **TD** or **pipeline** script handles it through a CLI tool:

```
python submit_render.py --scene /path/to/scene.mb --frames 1-100 --priority high
```

Frame range to render

Python script that sends job
info to the render farm

Scene file to be rendered
(Maya, Nuke, Houdini, etc.)

How urgently the job should be processed
(e.g. high, medium, low)

Why Do We Use CLI for Render Submission?

The main reason is **flexibility**.

The script logic stays the same, but the **inputs** change depending on:

- The **scene** you want to **render**
- The **number of frames**
- The **priority level**
- Maybe even the **software** (Maya, Nuke, Houdini)

You could reuse this one CLI tool like this:

```
python submit_render.py --scene /project/shots/shot01.mb --frames 1-50 --priority high  
python submit_render.py --scene /project/shots/shot02.nk --frames 51-100 --priority medium
```

Same script — different data **each time**. This makes the workflow **scalable** and **consistent**.

How Do Artists Use CLI Tools Without Typing Commands?

- **CLI tools are usually built for technical users**, like pipeline developers and TDs.
- But the same tools can be **triggered by artists through a user interface (UI)** — without them even knowing a script is being run.

Example Workflow: Render Submission from a UI

Let's say an artist is in **Maya**, and they open the **render submission panel**.

- The UI has:
 - A **file browser** to select the scene file
 - **Text inputs** for the frame range (e.g., 1-100)
 - A **dropdown** to set priority: High, Medium, Low
 - A "**Submit Render**" button

Artist



User interface

:

Collects inputs

CLI Tool

```
python submit_render.py -scene  
/path/to/scene.mb -frames 1-100 -priority high
```

Run Commands

Render Farm



✓ Render job
submitted successfully!

What Happens Behind the Scenes?

When the artist clicks **Submit Render**, the UI:

- Collects the inputs from the panel and Constructs the CLI command:

```
python submit_render.py --scene /path/to/scene.mb --frames 1-100 --priority high
```

Result:

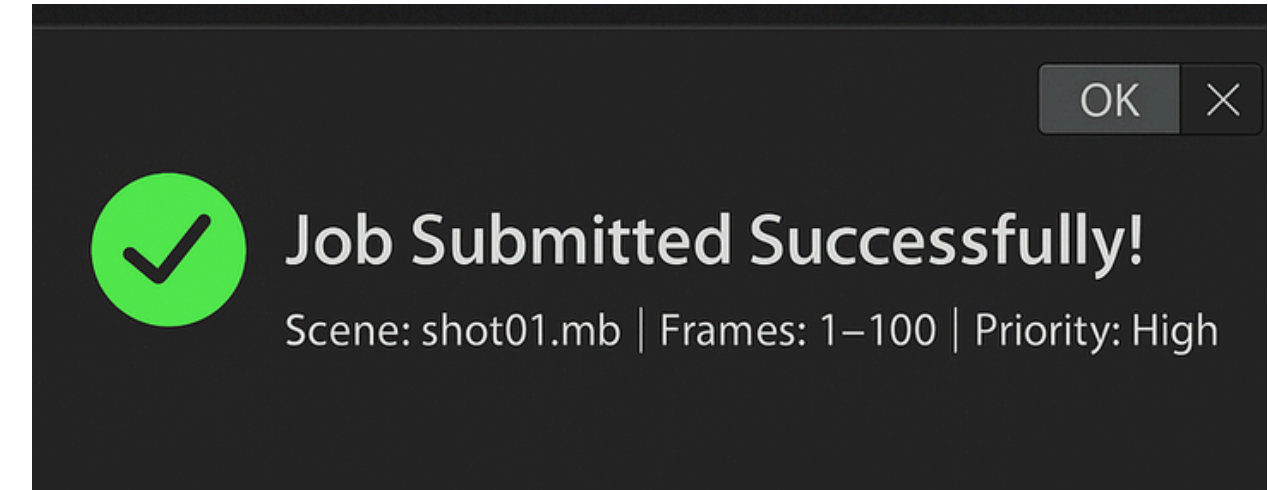
- Even though the artist never types that command
- the **UI collects the inputs** and passes them to the **CLI tool** under the hood.

How Does the Artist Know If the Job Was Submitted?

Once the job is sent to the farm, the system gives **feedback** in two ways:

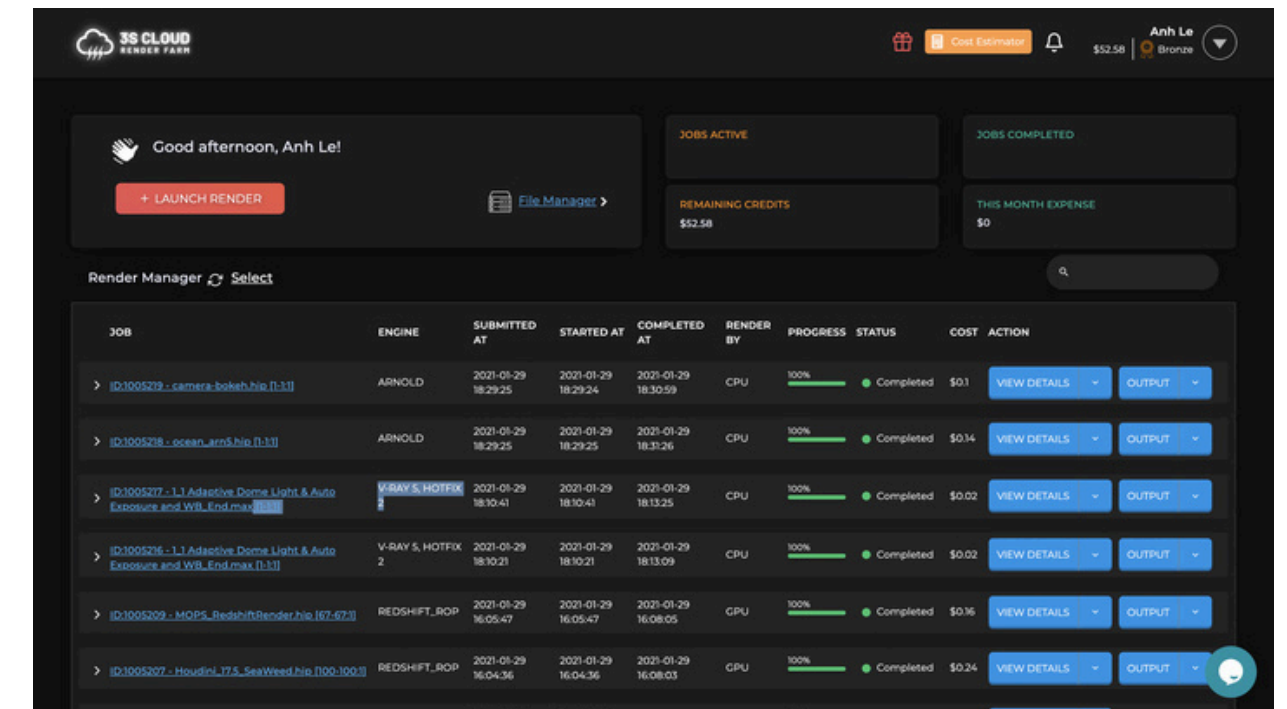
1. Immediate Confirmation

- A popup or message in the UI:
 - **"Job submitted successfully!"**



2. Farm Dashboard

- Artists can check the **status** in a **render farm tool**
 - **In Process** – Job is rendering
 - **Completed** – It's done!
 - **Failed** – Something went wrong (maybe a missing file or setting)



Why This Is Better

Feature

- **Reusable**
- **Flexible**
- **Easy to Automate**
- **UI Integration**

Benefit

- Same tool works for all shots and artists
 - Pass different inputs for different tasks
 - Perfect for pipeline automation, batch jobs, and studio-wide tasks
 - Artists never need to touch the command line — but it powers their tools
- 

Cleaning Up a Project Clutters

You've been working on a large VFX shot for several weeks.

- The project folder is now filled with cache files, logs, backups, and temp data from multiple tools.
- Manually deleting these is slow, error-prone, and tedious.

Instead, you run a CLI cleanup tool:

```
python cleanup.py --directory /path/to/project --types .tmp .cache --days 30
```

The root folder where cleanup
should happen

File extensions to target
e.g. .tmp, .cache, .log

Only delete files older than X days
e.g. 30 days old

Breaking Down the Command

Argument	What It Does
• cleanup.py	→ Python script that contains logic to scan and delete old files
• --directory	→ The root folder where cleanup should happen (/project/shot01/)
• --types	→ File extensions to target (e.g. .tmp, .cache, .log)
• --days	→ Only delete files older than X days (e.g. 30 days old)

Why Automate Project Cleanup?

Think about it:

- If you've been working on a project for weeks, the folder will likely be full of junk files.
 - Manually sorting through them is time-consuming and prone to error. This CLI tool:
 - **Saves time:** It's much faster than manually finding and deleting files.
 - **Keeps things clean:** Automates the process and ensures your workspace stays organized.
- 

Where Are CLI Tools Used?

Use Case

Why CLI Is Useful

- **Render Submission** → Quickly submit thousands of frames to a render farm with custom options
- **Asset Publishing** → Push finalized assets (models, textures, etc.) into version-controlled locations
- **Scene Validation / QC** → Run scripts that check for naming issues, missing textures, or bad geometry
- **Batch File Processing** → Convert image sequences, clean caches, copy files, or rename folders in bulk
- **Daily Automation** → Run scheduled scripts that generate playblasts, sync data, or prepare shots
- **Pipeline Integration** → Bridge artist tools with backend systems (like ShotGrid, ftrack, or versioning DBs)

Greeting Based on Time of Day

```
import sys

if len(sys.argv) < 2:
    print("Usage: python script.py <hour_in_24hr_format>")
else:
    hour = int(sys.argv[1])
    if 0 <= hour < 12:
        print("Good Morning!")
    elif 12 <= hour < 18:
        print("Good Afternoon!")
    elif 18 <= hour < 24:
        print("Good Evening!")
    else:
        print("Invalid hour! Please enter a value between 0 and 23.")
```

Run Command:

```
python script.py 9
```

Output:

```
Good Morning!
```

Basic Math Operations

```
import sys

if len(sys.argv) < 4:
    print("Usage: python script.py <number1> <operator> <number2>")
else:
    num1 = float(sys.argv[1])
    operator = sys.argv[2]
    num2 = float(sys.argv[3])
    if operator == '+':
        print(f"Result: {num1 + num2}")
    elif operator == '-':
        print(f"Result: {num1 - num2}")
    elif operator == '*':
        print(f"Result: {num1 * num2}")
    elif operator == '/':
        if num2 != 0:
            print(f"Result: {num1 / num2}")
        else:
            print("Error: Division by zero!")
    else:
        print("Invalid operator! Use +, -, *, or /.")
```

Run Command:

```
python script.py 10 + 5
```

Output:

```
Result: 15.0
```

Search in Text File

```
import sys

if len(sys.argv) < 3:
    print("Usage: python search_in_file.py <file_path> <keyword>")
else:
    file_path = sys.argv[1]
    keyword = sys.argv[2]
    try:
        with open(file_path, 'r') as file:
            lines = file.readlines()
            for line in lines:
                if keyword in line:
                    print(line.strip())
    except FileNotFoundError:
        print(f"Error: File '{file_path}' not found!")
```

File: sample.txt

```
Hello, this is a sample text file.
It contains several lines of text.
Some lines may include the keyword 'Python'.
Python is a powerful programming language.
CLI tools are also very useful.
```

Run Command:

```
python search_in_file.py sample.txt Python
```

Output:

```
Some lines may include the keyword 'Python'.
Python is a powerful programming language.
```

Count Words in File

```
import sys

if len(sys.argv) < 2:
    print("Usage: python word_count.py <file_path>")
else:
    file_path = sys.argv[1]
    try:
        with open(file_path, 'r') as file:
            content = file.read()
            words = content.split()
            print(f"The file contains {len(words)} words.")
    except FileNotFoundError:
        print(f"Error: File '{file_path}' not found!")
```

File: story.txt

```
Once upon a time, in a land far away, there lived a wise old man.
He loved to share stories and wisdom with everyone he met.
```

Run Command:

```
python word_count.py story.txt
```

Output:

```
The file contains 23 words.
```

Reverse Content of a File

```
import sys

if len(sys.argv) < 2:
    print("Usage: python reverse_lines.py <file_path>")
else:
    file_path = sys.argv[1]
    try:
        with open(file_path, 'r') as file:
            lines = file.readlines()
            for line in lines:
                print(line.strip()[::-1])
    except FileNotFoundError:
        print(f"Error: File '{file_path}' not found!")
```

File: quotes.txt

```
Never give up.
Dream big.
Work hard.
Stay humble.
```

Run Command:

```
python reverse_lines.py quotes.txt
```

Output:

```
.pu evig reven
.gib maerD
.drah krow
.elbmuh yatS
```

Rename Files in a Directory

```
import sys
import os

if len(sys.argv) < 2:
    print("Usage: python rename_files.py <directory_path>")
else:
    directory = sys.argv[1]

    try:
        for filename in os.listdir(directory):
            if filename.endswith(".txt"):
                old_path = os.path.join(directory, filename)
                new_path = os.path.join(directory, filename.replace(".txt", "_renamed.txt"))
                os.rename(old_path, new_path)
                print(f"Renamed: {filename} -> {new_path}")
    except FileNotFoundError:
        print(f"Error: Directory '{directory}' not found!")
```

Example Directory: files/

```
file1.txt
file2.txt
```

Run Command:

```
python rename_files.py files
```

Output:

```
Renamed: file1.txt -> files/file1_renamed.txt
Renamed: file2.txt -> files/file2_renamed.txt
```


Calculate Average Marks

```
import sys

if len(sys.argv) < 2:
    print("Usage: python average_marks.py <file_path>")
else:
    file_path = sys.argv[1]

    try:
        with open(file_path, 'r') as file:
            marks = [float(line.strip()) for line in file if line.strip()]
            if marks:
                average = sum(marks) / len(marks)
                print(f"Average Marks: {average:.2f}")
            else:
                print("No marks found in the file.")
    except FileNotFoundError:
        print(f"Error: File '{file_path}' not found!")
    except ValueError:
        print("Error: File contains invalid marks.")
```

File: marks.txt

85
78
92
88

Run Command:

```
python average_marks.py marks.txt

#Output: Average Marks: 85.75
```


Thank You

